

# Flipper Perps Aggregator

## Security Audit Report



July 10, 2026

BugBlow

Copyright © 2026 BugBlow. All rights reserved.

# Contents

|                                                                                                                   |           |
|-------------------------------------------------------------------------------------------------------------------|-----------|
| <b>1. INTRODUCTION</b>                                                                                            | <b>4</b>  |
| <b>1.1 Disclaimer</b>                                                                                             | 4         |
| <b>1.2 Security Assessment Methodology</b>                                                                        | 4         |
| 1.2.1 Code and architecture review:                                                                               | 4         |
| 1.2.2 Check code against vulnerability checklist and tools:                                                       | 4         |
| 1.2.3 Threat & Attack Simulation:                                                                                 | 4         |
| 1.2.4 Report found vulnerabilities:                                                                               | 5         |
| 1.2.5 Fix bugs & re-audit code:                                                                                   | 5         |
| 1.2.6 Deliver final report:                                                                                       | 5         |
| Vulnerability Risk Classification                                                                                 | 5         |
| <b>1.3 Project Overview</b>                                                                                       | 6         |
| <b>1.4 Project Dashboard</b>                                                                                      | 7         |
| Project Summary                                                                                                   | 7         |
| Project Audit Log                                                                                                 | 7         |
| Project Scope                                                                                                     | 7         |
| <b>1.5 Summary of findings</b>                                                                                    | 10        |
| Fix Commit Map                                                                                                    | 10        |
| <b>2. FINDINGS REPORT</b>                                                                                         | <b>12</b> |
| <b>2.1 Critical</b>                                                                                               | 12        |
| C-01: Treasury and insurance fees end up in vaults the withdraw instructions cannot reach                         | 12        |
| <b>2.2 High</b>                                                                                                   | 14        |
| H-01: Phoenix margin cannot be moved in or out of the DEX through the program                                     | 14        |
| H-02: Anyone can close the wrong Phoenix position of a user                                                       | 16        |
| H-03: Two positions pointing at the same Phoenix subaccount let a user withdraw against double-counted collateral | 19        |
| H-04: Builder rebates configured for Phoenix go nowhere                                                           | 22        |
| H-05: GMTrade order accounts are forwarded without checking who owns them                                         | 24        |
| H-06: Liquidation may lock a user's positions on the DEX forever                                                  | 27        |
| H-07: A leftover failed leg lets a user drain their vault while a position is still open                          | 29        |
| H-08: Collateral over-withdrawal through handle_withdraw                                                          | 31        |
| H-09: Collateral over-withdrawal through handle_withdraw_collateral                                               | 34        |
| H-10: Unbound collateral Pyth feed lets a user underpay protocol fees                                             | 37        |
| H-11: A partial close on Adrena is recorded as partial but the DEX has already flattened the whole position       | 38        |
| H-12: Phoenix close trusts the requested size instead of reading what actually filled                             | 41        |
| <b>2.3 Medium</b>                                                                                                 | 43        |
| M-01: Sour close lets the user redirect the proceeds away from the protocol vault                                 | 43        |
| M-02: Emergency close marks a position as closed while an async leg is still pending                              | 45        |

|                                                                                                                                  |           |
|----------------------------------------------------------------------------------------------------------------------------------|-----------|
| M-03: Phoenix legs report they consumed the entire account tail, breaking any compound trade where Phoenix is not last . . . . . | 49        |
| M-04: Withdraw silently sees zero Sour collateral and grief-blocks the user . . . . .                                            | 51        |
| M-05: Adrena close path lets the closer pick the rebate receiver . . . . .                                                       | 53        |
| M-06: USD close fee transferred as native token units, causing overcharge . . . . .                                              | 55        |
| M-07: Unit mismatch in margin check blocks legitimate withdrawals . . . . .                                                      | 57        |
| M-08: Cranker-supplied slippage overrides the protocol cap, enabling worst-fill stop-outs                                        | 58        |
| M-09: Partial fill finalized as fully closed in the async close branch . . . . .                                                 | 60        |
| M-10: Referral fee transfer breaks all non-USDC trades for referred users . . . . .                                              | 62        |
| <b>CONCLUSION</b>                                                                                                                | <b>64</b> |
| <b>ABOUT BUGBLOW</b>                                                                                                             | <b>65</b> |
| <b>Why BugBlow</b> . . . . .                                                                                                     | 65        |
| <b>Our Services</b> . . . . .                                                                                                    | 65        |
| <b>Contact Information</b> . . . . .                                                                                             | 65        |

# 1. INTRODUCTION

## 1.1 Disclaimer

The audit makes no statements or warranties about utility of the code, safety of the code, suitability of the business model, investment advice, endorsement of the platform or its products, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only. The information presented in this report is confidential and privileged. If you are reading this report, you agree to keep it confidential, not to copy, disclose or disseminate without the agreement of the Client. If you are not the intended recipient(s) of this document, please note that any disclosure, copying or dissemination of its content is strictly forbidden.

## 1.2 Security Assessment Methodology

BugBlow utilized a widely adopted approach to performing a security audit. Below is a breakout of how our team was able to identify and exploit vulnerabilities.

### 1.2.1 Code and architecture review:

- Review the source code.
- Read the project's documentation.
- Review the project's architecture.

Stage goals

- Build a good understanding of how the application works.

### 1.2.2 Check code against vulnerability checklist and tools:

- Understand the project's critical assets, resources, and security requirements.
- Manual check for vulnerabilities based on the auditor's checklist.
- Run the automated Savant Chat AI tool
- Run static analyzers.

Stage goals

- Identify and eliminate typical vulnerabilities (gas limit, replay, flash-loans attack, etc.)

### 1.2.3 Threat & Attack Simulation:

- Analyze the project's critical assets, resources and security requirements.
- Exploit the found weaknesses in a safe local environment.
- Document the performed work.

#### Stage goals

- Identify vulnerabilities that are not listed in static analyzers that would likely be exploited by hackers.
- Develop Proof of Concept (PoC).

#### 1.2.4 Report found vulnerabilities:

- Discuss the found issues with developers
- Show remediations.
- Write an initial audit report.

#### Stage goals

- Verify that all found issues are relevant.

#### 1.2.5 Fix bugs & re-audit code:

- Help developers fix bugs.
- Re-audit the updated code again.

#### Stage goals

- Double-check that the found vulnerabilities or bugs were fixed and were fixed correctly.

#### 1.2.6 Deliver final report:

- The Client deploys the re-audited version of the code
- The final report is issued for the Client.

#### Stage goals

- Provide the Client with the final report that serves as security proof.

### Vulnerability Risk Classification

All vulnerabilities discovered during the audit are classified based on a combination of their potential **impact** and **likelihood** of exploitation. The resulting severity is determined using the following risk matrix:

#### Severity Level Matrix

| Severity           | Impact: High | Impact: Medium | Impact: Low |
|--------------------|--------------|----------------|-------------|
| Likelihood: High   | Critical     | High           | Medium      |
| Likelihood: Medium | High         | Medium         | Low         |
| Likelihood: Low    | Medium       | Low            | Low         |

## Impact

- **High:** Direct theft or permanent loss exceeding 1% of the protocol's TVL, or permanent blocking of user funds (>1%) without the possibility of withdrawal or recovery.
- **Medium:** One-time theft of up to 1% of the protocol's TVL, or fund locking that can only be resolved through a contract upgrade, or indirect losses (for example lost yield, blocked rewards) exceeding 1% where eventual recovery is possible.
- **Low:** Contract malfunction or temporary lock that can be resolved by the administrator without a contract upgrade, with financial impact below 0.1% of TVL or negligible effect on end users.

## Likelihood

- **High:** Exploitation requires no privileged access and no rare preconditions. Any external actor can trigger the vulnerability under common contract states. Estimated probability of occurrence: >50% within a year.
- **Medium:** Exploitation requires specific but realistic conditions, such as a particular contract state, market movement, or trusted actor involvement. Estimated probability: 10 to 30% within a year.
- **Low:** Exploitation requires an extremely rare combination of conditions, a compromised privileged actor, or conditions with <5% probability of occurring within a year.

## Action Required

- **Critical**: Must be fixed immediately before deployment or as an emergency upgrade.
- **High**: It is strongly recommended to fix in the nearest release cycle.
- **Medium**: Recommended to fix to improve protocol security and stability.
- **Low**: Advisable to fix for overall robustness; low urgency.

## Finding Status

- **Open**: The issue has not been fixed yet.
- **Fixed**: The issue has been fixed.
- **Acknowledged**: The team has reviewed the finding but has not yet implemented changes, or the finding does not require code modifications.

# 1.3 Project Overview

Flipper is an on-chain perpetuals aggregator built on Solana with the Anchor framework. Instead of running its own order book, it splits a single user order across several upstream perpetual DEXes (Adrena, GMTrade, Phoenix, Sour, and previously Drift, Jupiter Perps, Flash) in one atomic transaction. User collateral lives in protocol-owned vault PDAs (a USDC vault per user, plus optional per-mint vaults for non-USDC collateral such as SOL, wBTC, wETH and USDT), and the program signs cross-program calls into each upstream DEX on the user's behalf. The on-chain program owns the user vaults, the aggregated cross-DEX position state, the order intents, protocol configuration (fees, leverage and slippage caps, oracle staleness, halt switch), the treasury, the insurance fund, settlement

state and per-DEX circuit breakers. Price feeds come from the Pyth oracle.

## 1.4 Project Dashboard

### Project Summary

| Title              | Flipper Perps Aggregator |
|--------------------|--------------------------|
| Client name        | Flipper                  |
| Project name       | Flipper Perps Aggregator |
| Timeline           | 11.05.2026 - 06.07.2026  |
| Number of Auditors | 2                        |

### Project Audit Log

| Commit before audit                      | Commit after audit                       |
|------------------------------------------|------------------------------------------|
| 4966aa967978edbb9c0f1b6c888025873a37d127 | 1ee42fe3aeb4eade9e1b5fc7cad86fa0084154eb |

### Project Scope

The audit covered the following smart contract files.

| File name                                                               |
|-------------------------------------------------------------------------|
| programs/flipper-core/src/instructions/liquidate_position.rs            |
| programs/flipper-core/src/instructions/handle_liquidation.rs            |
| programs/flipper-core/src/instructions/withdraw.rs                      |
| programs/flipper-core/src/instructions/withdraw_collateral.rs           |
| programs/flipper-core/src/instructions/withdraw_treasury.rs             |
| programs/flipper-core/src/instructions/withdraw_insurance_fund.rs       |
| programs/flipper-core/src/instructions/execute_trade_with_collateral.rs |
| programs/flipper-core/src/instructions/execute_close_with_collateral.rs |
| programs/flipper-core/src/instructions/execute_stop_out.rs              |
| programs/flipper-core/src/instructions/confirm_async_fill.rs            |
| programs/flipper-core/src/instructions/timeout_async_fill.rs            |
| programs/flipper-core/src/instructions/execute_compound_trade.rs        |

| File name                                                               |
|-------------------------------------------------------------------------|
| programs/flipper-core/src/instructions/execute_compound_close.rs        |
| programs/flipper-core/src/instructions/emergency_close_sub_position.rs  |
| programs/flipper-core/src/instructions/register_collateral_mint.rs      |
| programs/flipper-core/src/helpers/margin.rs                             |
| programs/flipper-core/src/state/aggregated_position.rs                  |
| programs/flipper-core/src/state/collateral.rs                           |
| programs/flipper-core/src/lib.rs                                        |
| programs/flipper-core/src/constants.rs                                  |
| programs/flipper-core/src/errors.rs                                     |
| programs/flipper-core/src/events.rs                                     |
| programs/flipper-core/src/instructions/create_order_intent.rs           |
| programs/flipper-core/src/instructions/cancel_order.rs                  |
| programs/flipper-core/src/instructions/force_cancel_orders.rs           |
| programs/flipper-core/src/instructions/update_position.rs               |
| programs/flipper-core/src/instructions/execute_trigger_order.rs         |
| programs/flipper-core/src/instructions/trigger_global_settlement.rs     |
| programs/flipper-core/src/instructions/settle_position.rs               |
| programs/flipper-core/src/instructions/cancel_global_settlement.rs      |
| programs/flipper-core/src/instructions/deposit.rs                       |
| programs/flipper-core/src/instructions/deposit_collateral.rs            |
| programs/flipper-core/src/instructions/init_collateral_vault.rs         |
| programs/flipper-core/src/instructions/initialize_user.rs               |
| programs/flipper-core/src/instructions/initialize_protocol_config.rs    |
| programs/flipper-core/src/instructions/update_protocol_config.rs        |
| programs/flipper-core/src/instructions/initialize_market_registry.rs    |
| programs/flipper-core/src/instructions/update_market_registry.rs        |
| programs/flipper-core/src/instructions/initialize_dex_market_mapping.rs |
| programs/flipper-core/src/instructions/initialize_treasury.rs           |
| programs/flipper-core/src/instructions/initialize_insurance_fund.rs     |
| programs/flipper-core/src/instructions/initialize_insurance_vault.rs    |



| File name                                                              |
|------------------------------------------------------------------------|
| programs/flipper-core/src/instructions/initialize_builder_registry.rs  |
| programs/flipper-core/src/instructions/update_builder_entry.rs         |
| programs/flipper-core/src/instructions/initialize_circuit_breaker.rs   |
| programs/flipper-core/src/instructions/trip_circuit_breaker.rs         |
| programs/flipper-core/src/instructions/admin_trip_circuit_breaker.rs   |
| programs/flipper-core/src/instructions/reset_circuit_breaker.rs        |
| programs/flipper-core/src/instructions/set_admin.rs                    |
| programs/flipper-core/src/adapters/drift_adapter.rs                    |
| programs/flipper-core/src/adapters/gmtrade_adapter.rs                  |
| programs/flipper-core/src/adapters/adrena_adapter.rs                   |
| programs/flipper-core/src/adapters/phoenix_adapter.rs                  |
| programs/flipper-core/src/adapters/sour_adapter.rs                     |
| programs/flipper-core/src/adapters/flash_adapter.rs                    |
| programs/flipper-core/src/adapters/jupiter_adapter.rs                  |
| programs/flipper-core/src/adapters/custody.rs                          |
| programs/flipper-core/src/adapters/cpi_interfaces/drift.rs             |
| programs/flipper-core/src/adapters/cpi_interfaces/gmtrade.rs           |
| programs/flipper-core/src/adapters/cpi_interfaces/adrena.rs            |
| programs/flipper-core/src/adapters/cpi_interfaces/phoenix.rs           |
| programs/flipper-core/src/adapters/cpi_interfaces/sour.rs              |
| programs/flipper-core/src/adapters/cpi_interfaces/flash.rs             |
| programs/flipper-core/src/adapters/cpi_interfaces/jupiter.rs           |
| programs/flipper-core/src/adapters/cpi_interfaces/discriminator_pin.rs |
| programs/flipper-core/src/state/order_intent.rs                        |
| programs/flipper-core/src/state/protocol_config.rs                     |
| programs/flipper-core/src/state/market_registry.rs                     |
| programs/flipper-core/src/state/dex_market_mapping.rs                  |
| programs/flipper-core/src/state/dex_circuit_breaker.rs                 |
| programs/flipper-core/src/state/builder_registry.rs                    |
| programs/flipper-core/src/state/insurance_fund.rs                      |

| File name                                            |
|------------------------------------------------------|
| programs/flipper-core/src/state/settlement_state.rs  |
| programs/flipper-core/src/state/treasury.rs          |
| programs/flipper-core/src/state/user_account.rs      |
| programs/flipper-core/src/helpers/oracle.rs          |
| programs/flipper-core/src/helpers/circuit_breaker.rs |

## 1.5 Summary of findings

Findings chart

| Severity        | # of Findings |
|-----------------|---------------|
| <b>Critical</b> | 1             |
| <b>High</b>     | 12            |
| <b>Medium</b>   | 10            |
| <b>Low</b>      | 0             |

All 23 findings have been fixed on the branch. The remediation was verified against the **fix/mainnet** head commit listed in the Project Audit Log.

### Fix Commit Map

Each finding was closed by one or more commits on **fix/mainnet**. The table below lists the primary commit for every finding; secondary commits are mentioned inline where relevant.

| ID   | Primary fix commit                                                                                                          | Short SHA        |
|------|-----------------------------------------------------------------------------------------------------------------------------|------------------|
| C-01 | fix(admin): FLIP2-1 withdraw treasury/insurance from per-mint vaults; hardened by fix(vaults) M1 v2 per-mint fee accounting | b998f80, 7834bc2 |
| H-01 | feat(phoenix): mainnet-faithful collateral lifecycle + fork e2e proof                                                       | 03dbc14          |
| H-02 | fix(security): remediate mainnet aggregator audit 2026-06 (H-02/H-04/H-05/M-02/M-05)                                        | 85d2b93          |
| H-03 | fix(program): PHX-3 caller-supplied Phoenix subaccount_index + isolated-margin withdraw refactor                            | a311731          |
| H-04 | fix(security): Flight proxy wrapping for Phoenix builder rebate                                                             | 85d2b93          |

| ID   | Primary fix commit                                                                                | Short SHA |
|------|---------------------------------------------------------------------------------------------------|-----------|
| H-05 | fix(security): GMTrade escrow ATA pin plus None-sentinel slots                                    | 85d2b93   |
| H-06 | fix(audit): remove liquidate_position (redundant unsafe shortcut)                                 | d1243d1   |
| H-07 | fix(handle_liquidation): aggregate active-status gate before sub work                             | d1243d1   |
| H-08 | fix(withdraw): retire USDC-only handle_withdraw in favour of unified collateral path              | d1243d1   |
| H-09 | fix(withdraw_collateral): status-filter, position_id dedup, single aggregate meets_initial_margin | d1243d1   |
| H-10 | fix(oracle): bind collateral Pyth Pull v2 feed to CollateralType.pyth_feed_id()                   | d1243d1   |
| H-11 | fix(dex): reconcile partial close accounting (Adrena close_percent)                               | 22f2af5   |
| H-12 | fix(dex): reconcile partial close accounting (Phoenix realized-size)                              | 22f2af5   |
| M-01 | fix(sour): FLIP2-15 pin the pooled vault USDC ATA by its SPL authority                            | 8c657e4   |
| M-02 | fix(security): emergency close guarded by has_inflight_sub_position                               | 85d2b93   |
| M-03 | feat(core): multi-DEX router per-leg slicing + leg_count assertion                                | d1243d1   |
| M-04 | closed by the isolated-margin withdraw refactor (no cross-leg custody sum remains)                | a311731   |
| M-05 | fix(security): BuilderEntry.referrer_profile + Adrena pin_referrer_profile helper                 | 85d2b93   |
| M-06 | feat(flipper-core): close-path bundle per-mint fee vaults + USD-6 scaling (FLIP 1-17)             | d4729aa   |
| M-07 | fix(withdraw_collateral): native to USD-6 normalisation via collateral Pyth feed (FLIP 1-11)      | d1243d1   |
| M-08 | fix(execute_stop_out): cap params.max_slippage_bps.min(config.max_slippage_bps)                   | d1243d1   |
| M-09 | fix(confirm_async_fill): cumulative closed_size_usd with PartialClosed transition                 | d1243d1   |
| M-10 | fix(referrals): per-(referrer, mint) referrer vault + per-mint validation on open/close           | 871b190   |

## 2. FINDINGS REPORT

### 2.1 Critical

C-01: Treasury and insurance fees end up in vaults the withdraw instructions cannot reach

|                 |                                                                                     |
|-----------------|-------------------------------------------------------------------------------------|
| <b>C-01</b>     | Treasury and insurance fees end up in vaults the withdraw instructions cannot reach |
| <b>Severity</b> | <b>Critical</b>                                                                     |
| <b>Status</b>   | <b>Fixed</b>                                                                        |
| <b>Location</b> | withdraw_treasury.rs, withdraw_insurance_fund.rs                                    |

#### Description

Likelihood: High. Every protocol fee, on every trade, ends up in the wrong place. Impact: High. The protocol cannot withdraw any of its accumulated revenue.

Fees are paid into one vault per collateral mint. The treasury vault is derived from `seeds = [TREASURY_VAULT_SEED, mint]` and the insurance vault from `seeds = [INSURANCE_VAULT_SEED, mint]`. This is how `execute_trade_with_collateral`, `execute_close_with_collateral`, `execute_stop_out`, `execute_trigger_order`, `execute_limit_order_intent` and `confirm_stop_out_close` all route their fee transfers.

But the two withdraw instructions target the older singleton vaults, derived without the mint segment:

```
// withdraw_treasury.rs
#[account(seeds = [TREASURY_VAULT_SEED], bump)]
pub treasury_token_account: Account<'info, TokenAccount>,

// withdraw_insurance_fund.rs
#[account(seeds = [INSURANCE_VAULT_SEED], bump)]
pub insurance_vault: Account<'info, TokenAccount>,
```

These singleton vaults are created at initialisation and then never receive a single deposit. Their balance is permanently zero. The treasury counter that tracks "how much we have collected" keeps showing the real total, but the actual money sits in the per-mint vaults with no instruction that can sign for them.

For the treasury instruction, the cap calculation collapses to zero:

```
let max_withdrawable = params.amount.min(net_balance).min(vault_balance);
// vault_balance = 0
require!(params.amount <= max_withdrawable, InsufficientBalance);
```

So every call reverts. For the insurance instruction the cap is only against the counter, so the require

check passes, and then the token transfer itself reverts because the source vault is empty.

This is the same bug class as the original Hexens FLIP 1-14 finding, where fees were collected with no withdrawal path. The fix at the time added the counter and the withdraw instruction, but the later migration to per-mint vaults silently re-armed it. Every accrual moved to per-mint vaults, and the withdraw instructions were not updated. The internal doc comment admits this is FLIP 1-17 follow-up work that has not shipped.

Scenario:

1. The protocol registers USDC and WBTC as collateral mints. Per-mint vaults are created for each.
2. A user trades with WBTC collateral. The fee is scaled into about 1500 WBTC base units and transferred to the WBTC treasury vault. The counter goes up by the dollar value.
3. The admin tries to withdraw. The instruction signs for the singleton vault, finds it empty, reverts.
4. Every subsequent trade adds more to the per-mint vaults. The admin has no way to take any of it out. Same shape for insurance.

### Suggested Fix

Make both withdraw instructions per-mint:

1. Add a 'mint' account input.
2. Constrain the vault as seeds = [TREASURY\_VAULT\_SEED, mint.key().as\_ref()] and the same shape for insurance.
3. Track accumulated and withdrawn amounts per mint, not as a single global USD counter. Mixing dollars on the counter side with native units on the transfer side gives the wrong cap.

If a mainnet deploy has to happen before the above, route every fee back to the singleton vault until the per-mint withdraw instructions are ready.

### Proof of Concept

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_singleton_vault_differs_from_per_mint_vaults() {
        let program_id = Pubkey::new_unique();
        let usdc_mint = Pubkey::new_unique();
        let wbtc_mint = Pubkey::new_unique();
```

```

        // What 'withdraw_treasury' signs for.
        let (singleton, _) =
            Pubkey::find_program_address(&[TREASURY_VAULT_SEED], &
program_id);

        // What every accrual actually deposits into.
        let (per_mint_usdc, _) = Pubkey::find_program_address(
            &[TREASURY_VAULT_SEED, usdc_mint.as_ref()], &program_id);
        let (per_mint_wbtc, _) = Pubkey::find_program_address(
            &[TREASURY_VAULT_SEED, wbtc_mint.as_ref()], &program_id);

        assert_ne!(singleton, per_mint_usdc);
        assert_ne!(singleton, per_mint_wbtc);
    }

    #[test]
    fn test_withdraw_cap_collapses_to_zero_because_singleton_is_empty() {
        // Replay the cap math from the handler.
        let counter_balance: u64 = 100_000_000_000; // $100k tracked
        let singleton_balance: u64 = 0;             // never receives
fees
        let requested: u64 = 50_000_000_000;

        let max_withdrawable = requested.min(counter_balance).min(
singleton_balance);
        assert_eq!(max_withdrawable, 0);
        // require!(requested <= max_withdrawable) reverts for any
positive amount.
        assert!(requested > max_withdrawable);
    }
}

```

## 2.2 High

H-01: Phoenix margin cannot be moved in or out of the DEX through the program

|                 |                                                                         |
|-----------------|-------------------------------------------------------------------------|
| <b>H-01</b>     | Phoenix margin cannot be moved in or out of the DEX through the program |
| <b>Severity</b> | High                                                                    |
| <b>Status</b>   | Fixed                                                                   |
| <b>Location</b> | phoenix_adapter.rs, cpi_interfaces/phoenix.rs                           |

## Description

Likelihood: Medium. The exposure depends on Phoenix routing being live, which the protocol owns the timing of. Impact: High. Once Phoenix is enabled, any margin or profit on the DEX side has no way back to the user's vault.

Phoenix stores each trader's margin inside a small per-trader account on the DEX. The only way money can enter or leave that account is through a CPI signed by Flipper's per-user vault. The protocol declares the three Phoenix instructions for this lifecycle (**register\_trader**, **deposit\_funds**, **withdraw\_funds**) and pins their identifiers in tests, but it never actually calls any of them anywhere in the codebase. The only Phoenix CPI Flipper performs is the market order itself.

The adapter doc comment says the subaccount is "assumed to be already registered and USDC-funded" and that funding happens "as separate lifecycle steps". Those steps do not exist as Flipper instructions, on either the audited snapshot or the current branch head.

What this means in practice:

1. A trader opens a Phoenix position. The market order requires the subaccount to already hold USDC margin. With no Flipper-side deposit, the subaccount is empty, and the DEX rejects the order. Every Phoenix open reverts.
2. Suppose funding is done somehow off-chain anyway. After a close, the order returns money back to the subaccount, not to the user's vault. Without a withdraw call, that money has no path back to Flipper. It sits permanently locked inside the Phoenix subaccount.
3. The close-side fee block silently skips collecting fees because it compares the scaled fee against the user's vault balance, which never received the close proceeds.

## Locations

- `programs/flipper-core/src/adapters/phoenix_adapter.rs`: L5 to 8 (doc comment claims "separate lifecycle steps")
- `programs/flipper-core/src/adapters/phoenix_adapter.rs`: L99 to 116 (the only CPI that exists)
- `programs/flipper-core/src/adapters/cpi_interfaces/phoenix.rs`: L204 to 216 (lifecycle identifiers defined but unused)

## Suggested Fix

Two paths, either is acceptable; the long-term solution is the first one.

1. Ship the missing Flipper instructions:
  - One that bundles 'register\_trader' plus 'deposit\_funds' so a new sub-position funds the Phoenix subaccount in a single transaction signed by the vault.
  - A 'withdraw\_funds' CPI folded into the close path so margin and PnL flow back to the vault inside the same close transaction.
2. If a mainnet deploy needs to happen before the above, disable Phoenix routing entirely at the dispatch boundary while the build targets

mainnet. Otherwise any Phoenix trade either reverts or strands user funds.

## Proof of Concept

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_phoenix_lifecycle_discriminators_defined_but_never_invoked()
    {
        use crate::adapters::cpi_interfaces::phoenix::{
            PHOENIX_DEPOSIT_FUNDS_DISCRIMINATOR,
            PHOENIX_REGISTER_TRADER_DISCRIMINATOR,
            PHOENIX_WITHDRAW_FUNDS_DISCRIMINATOR,
        };

        // The three lifecycle identifiers exist as constants.
        assert_eq!(PHOENIX_REGISTER_TRADER_DISCRIMINATOR.len(), 8);
        assert_eq!(PHOENIX_DEPOSIT_FUNDS_DISCRIMINATOR.len(), 8);
        assert_eq!(PHOENIX_WITHDRAW_FUNDS_DISCRIMINATOR.len(), 8);

        // But the only CPI helper the adapter exposes is the market
        // order.
        // No 'deposit_funds' or 'withdraw_funds' helper exists. A fix
        // must
        // add new public helpers, which would change this surface.
        let _market_order: fn(super::OrderPacket, &[AccountInfo], &[u8
    ])
        -> Result<()> = place_market_order;
    }
}
```

H-02: Anyone can close the wrong Phoenix position of a user

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| <b>H-02</b>     | Anyone can close the wrong Phoenix position of a user |
| <b>Severity</b> | High                                                  |
| <b>Status</b>   | Fixed                                                 |
| <b>Location</b> | phoenix_adapter.rs: L51-74, L120-232                  |



## Description

Likelihood: Medium. The attack needs the user to hold more than one Phoenix position on the same collateral mint. Impact: High. The attacker can flatten one of the user's positions at terms the user did not consent to.

Phoenix separates each trader's positions into subaccounts. A subaccount is owned by a single authority key, and on Flipper the authority is the per-user vault. Two of the user's Phoenix positions share the same authority and are distinguished only by a subaccount index passed in the account list.

The Phoenix adapter validates that the signer at index 3 is the correct vault, but it never checks that the actual position account at index 4 is the one this close is supposed to act on. The expected position account is stored on Flipper's side as **sub.dex\_position\_account**, but it is not even passed into the close function, let alone compared against the supplied account.

So if a permissionless instruction like **execute\_trigger\_order** or **execute\_stop\_out** fires for position A, anyone who runs the crank can swap in the subaccount belonging to position B. Phoenix accepts it because the authority is correct. The order closes position B at the slippage band and oracle price that were computed for A.

The adapter comment claims this is safe because "proceeds always land in the subaccount owned by that authority". That is true and not the point. The attack is closing the wrong subaccount, not redirecting the money.

Scenario:

1. The user opens position A with a Phoenix leg at subaccount index 1, and position B at subaccount index 2. Both are SOL-PERP. Position B is much larger.
2. The price reaches A's take-profit. A keeper runs **execute\_trigger\_order** for position A.
3. The keeper builds the account list with position B's subaccount at index 4. The genuine subaccount for A is placed elsewhere in the trailing accounts so the health check that scans by key still finds it.
4. The Phoenix adapter only checks the signer. Phoenix's own check, "subaccount authority equals signer", passes for B as well. The reduce-only order hits position B and flattens it.
5. Flipper marks position A as closed and credits the realised PnL using A's parameters. Position B is now silently closed on the DEX. Flipper still thinks B is open.

A related variant happens at open time. The open path records whatever account is at index 4 as **sub.dex\_position\_account**. Nothing forces this to be the canonical subaccount for the new position. A user opening their second Phoenix position can deliberately re-use the first position's subaccount, and Flipper will record both positions as pointing at the same DEX-side state. This becomes a money bug through finding H-03 below.

## Locations

- **programs/flipper-core/src/adapters/phoenix\_adapter.rs**: L51 to 74 (signer check at index 3 only)
- **programs/flipper-core/src/adapters/phoenix\_adapter.rs**: L177 to 232 (close path, no index 4 check)

- `programs/flipper-core/src/adapters/phoenix_adapter.rs`: L120 to 173 (open path records index 4 unchecked)
- `execute_trigger_order.rs` and `execute_stop_out.rs` (callers that do not pass the expected position)

## Suggested Fix

On the close, trigger and stop-out paths:

1. Add an `'expected_position: Pubkey'` argument to `'phoenix_adapter::close_position'`.
2. Pass `'sub.dex_position_account'` from every caller.
3. Inside the adapter, require `'remaining_accounts[4].key() == expected_position'`.

On the open path, do the same in reverse: derive the canonical Phoenix subaccount from `'(vault, market, subaccount_index)'` where `'subaccount_index'` is chosen deterministically per Flipper position (for example `'position_id mod N'`), and require index 4 to match. Track the chosen index on the position so future opens do not collide.

## Proof of Concept

```
#[cfg(test)]
mod tests {
    use super::*;
    use crate::constants::COLLATERAL_VAULT_SEED;

    #[test]
    fn test_two_phoenix_positions_share_the_same_vault_authority() {
        // The bug exists because both of a user's Phoenix positions
        share
        // the col_vault authority. Phoenix tells them apart only by the
        // subaccount byte, and Flipper does not check it.
        let user = Pubkey::new_unique();
        let mint = Pubkey::new_unique();
        let market = Pubkey::new_unique();
        let phoenix_program = Pubkey::new_unique();

        let (col_vault, _) = Pubkey::find_program_address(
            &[COLLATERAL_VAULT_SEED, user.as_ref(), mint.as_ref()],
            &crate::ID,
        );

        let sub = |idx: u8| -> Pubkey {
            Pubkey::find_program_address(
                &[b"trader_subaccount", col_vault.as_ref(), market.as_ref()],
                &[idx]],
            )
        }
```

```

        &phoenix_program,
    ).0
};

let sub_1 = sub(1);
let sub_2 = sub(2);
assert_ne!(sub_1, sub_2, "different subaccounts as expected");
}

#[test]
fn test_close_signature_has_no_room_for_expected_position() {
    // Pin the missing input. Any fix that adds the expected_position
    // argument changes this function pointer type and breaks the
    test.
    let _close_fp: fn(
        u64,
        crate::state::order_intent::Side,
        crate::state::order_intent::MarketType,
        u16,
        u64,
        &[AccountInfo],
        &[&[u8]],
    ) -> Result<CpiCloseResult> = close_position;
}
}

```

H-03: Two positions pointing at the same Phoenix subaccount let a user withdraw against double-counted collateral

|                 |                                                                                                             |
|-----------------|-------------------------------------------------------------------------------------------------------------|
| <b>H-03</b>     | Two positions pointing at the same Phoenix subaccount let a user withdraw against double-counted collateral |
| <b>Severity</b> | <b>High</b>                                                                                                 |
| <b>Status</b>   | <b>Fixed</b>                                                                                                |
| <b>Location</b> | withdraw_collateral.rs, phoenix_adapter.rs                                                                  |

### Description

Likelihood: Medium. Requires the user to deliberately reuse a Phoenix subaccount across two of their positions, which is possible through H-02. Impact: High. Direct over-withdraw from the vault, backed by collateral the protocol does not actually have.

When the user withdraws non-USDC collateral, the program adds up effective collateral from two

sources: what is left in the per-mint vault, and what the DEXes are currently holding for each of the user's open legs. The DEX side is read inline by calling **decode\_leg\_custody** for each open leg, which reads the position account and returns the collateral it holds.

The instruction protects against passing the same position twice by tracking **seen\_position\_ids**:

```
require!(
    !seen_position_ids.contains(&pos.position_id),
    FlipperError::InvalidRemainingAccounts
);
seen_position_ids.push(pos.position_id);
```

This dedup is keyed only on the position id, not on the DEX-side account each position points at. Two positions with different ids can share the same **sub.dex\_position\_account** (this is exactly what H-02's open-time variant produces for Phoenix). Both pass the dedup, both call **decode\_leg\_custody** against the same Phoenix subaccount, and both add its balance to the effective-collateral total.

The total now claims twice the real Phoenix collateral. The maintenance-margin check passes against that inflated number, and the user is allowed to draw vault funds that should still be locked against real exposure.

Scenario:

1. The user deposits \$100k USDC and opens position 1 entirely on Phoenix with no leverage. All \$100k flows into Phoenix subaccount A. The per-mint vault residual is zero.
2. The user opens position 2 also on Phoenix, but supplies subaccount A again at index 4. Per H-02 the open path accepts it and stamps **sub.dex\_position\_account = A** on position 2 as well.
3. The user has another small open position elsewhere (say Adrena) for \$20k, sitting in the vault.
4. The user calls **withdraw\_collateral** and lists both Phoenix positions plus the Adrena one. Dedup sees three distinct ids and passes.
5. Effective collateral comes out to vault residual, plus subaccount A read once for position 1, plus subaccount A read again for position 2, plus Adrena. The Phoenix piece is double-counted.
6. The maintenance-margin check sees plenty of cover and approves a \$90k withdraw. The Adrena leg is still open. Real collateral on Phoenix is \$100k, not \$200k. The protocol is now under-collateralised against that exposure.

## Locations

- **programs/flipper-core/src/instructions/withdraw\_collateral.rs**: L177 to 215 (dedup by position id only)
- **programs/flipper-core/src/instructions/withdraw\_collateral.rs**: L254 to 260 and L290 plus (DEX collateral aggregator)
- **programs/flipper-core/src/adapters/phoenix\_adapter.rs**: L120 to 173 (open path that does not pin index 4)

## Suggested Fix

Two layers, apply both for defence in depth.

1. Fix the open side so two positions cannot point at the same Phoenix subaccount. Derive the expected subaccount from `'(vault, market, subaccount_index)'` where the index is chosen per Flipper position, and require it to match at open.
2. Add a second dedup on the withdraw side, across all open legs of all positions:

```
let mut seen_dex_accounts: Vec<Pubkey> = Vec::new();
for sub in pos.active_sub_positions() {
    if sub.is_open() {
        require!(
            !seen_dex_accounts.contains(&sub.dex_position_account),
            FlipperError::DuplicateDexPositionAccount
        );
        seen_dex_accounts.push(sub.dex_position_account);
        dex_collateral_usd_u128 = dex_collateral_usd_u128
            .saturating_add(decode_leg_custody(...)?.collateral_usd as
                u128);
    }
}
```

## Proof of Concept

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_dedup_is_keyed_only_on_position_id() {
        let pos_id_1: u64 = 1;
        let pos_id_2: u64 = 2;
        let shared_subaccount = Pubkey::new_unique();

        let mut seen_position_ids: Vec<u64> = Vec::new();
        for id in [pos_id_1, pos_id_2] {
            assert!(!seen_position_ids.contains(&id));
            seen_position_ids.push(id);
        }
        assert_eq!(shared_subaccount, shared_subaccount);
    }

    #[test]
    fn test_collateral_aggregator_double_counts_the_shared_subaccount() {
        let subaccount_collateral_usd: u64 = 100_000_000_000; // $100k
        let mut dex_collateral_usd_u128: u128 = 0;
```

```

    // Iteration for position 1: reads subaccount, credits $100k.
    dex_collateral_usd_u128 = dex_collateral_usd_u128
      .saturating_add(subaccount_collateral_usd as u128);
    // Iteration for position 2: same subaccount, credits $100k again
    .
    dex_collateral_usd_u128 = dex_collateral_usd_u128
      .saturating_add(subaccount_collateral_usd as u128);

    // Books say $200k. Real Phoenix-side balance is $100k.
    assert_eq!(dex_collateral_usd_u128, 200_000_000_000);
  }
}

```

#### H-04: Builder rebates configured for Phoenix go nowhere

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <b>H-04</b>     | Builder rebates configured for Phoenix go nowhere |
| <b>Severity</b> | High                                              |
| <b>Status</b>   | Fixed                                             |
| <b>Location</b> | adapters/mod.rs, phoenix_adapter.rs               |

##### Description

Likelihood: High once an admin enables a Phoenix builder entry. Impact: Medium. The registered builder receives no rebates. Off-chain accounting that trusts the event stream diverges from on-chain reality.

The protocol has a builder registry that maps each DEX to a builder pubkey and a rebate share. The Phoenix slot in this registry is explicitly reserved as "the Flight builder-routing fee target", and **resolve\_builder\_target** returns a fully populated **BuilderTarget** when Phoenix has an active entry.

The dispatcher passes this target into the Adrena, GMTrade and Sour adapters. For Phoenix it does not, because Phoenix's **open\_position** and **close\_position** functions have no **builder** parameter at all. The resolved target gets dropped at the dispatch boundary.

The effect is a silent contract mismatch. An admin enables a Phoenix builder, sees the **BuilderEntryUpdated** event, watches off-chain attribution flow, and the rebate is never actually paid to the configured builder on chain. It stays with the Phoenix-default routing.

## Locations

- `programs/flipper-core/src/adapters/mod.rs`: L213 to 231 (Phoenix arm of `dispatch_open_position`)
- `programs/flipper-core/src/adapters/mod.rs`: L249 to 267 (Phoenix arm of `dispatch_close_position`)
- `programs/flipper-core/src/adapters/phoenix_adapter.rs`: L120 to 129 (open signature has no `builder`)
- `programs/flipper-core/src/adapters/phoenix_adapter.rs`: L177 to 185 (close signature has no `builder`)
- `programs/flipper-core/src/state/builder_registry.rs`: L37 to 39 (Phoenix slot reserved as Flight target)

## Suggested Fix

Pick one:

1. Plumb the builder through to Phoenix. Add `'builder: Option<BuilderTarget>'` to `'phoenix_adapter::open_position'` and `'close_position'`, and wire it into the Flight builder-target slot of the IOC packet. The Flight program id is already defined in `'constants.rs'`.
2. Block `'update_builder_entry'` from enabling the Phoenix slot until the above lands, so the admin cannot emit an event that has no on-chain effect.

## Proof of Concept

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_phoenix_builder_passes_filter_but_signature_drops_it() {
        // Step 1. Build a Phoenix builder target the way an admin would.
        let target = BuilderTarget {
            dex_id: DexType::Phoenix as u8,
            builder_pubkey: Pubkey::new_unique(),
            rebate_bps_of_notional: 15,
        };

        // Step 2. Dispatcher filter passes because the dex_id matches.
        let after_filter = Some(target).filter(|b| b.dex_id == DexType::Phoenix as u8);
        assert!(after_filter.is_some());
    }
}
```

```

        // Step 3. But the Phoenix open and close adapters have no '
builder'
        // parameter. The target is dropped at this boundary.
        let _open_fp: fn(
            u64, u8,
            crate::state::order_intent::Side,
            crate::state::order_intent::MarketType,
            u16, u64,
            &[AccountInfo],
            &[u8]),
        ) -> Result<CpiOpenResult> = phoenix_adapter::open_position;

        let _close_fp: fn(
            u64,
            crate::state::order_intent::Side,
            crate::state::order_intent::MarketType,
            u16, u64,
            &[AccountInfo],
            &[u8]),
        ) -> Result<CpiCloseResult> = phoenix_adapter::close_position;
    }
}

```

H-05: GMTrade order accounts are forwarded without checking who owns them

|                 |                                                                     |
|-----------------|---------------------------------------------------------------------|
| <b>H-05</b>     | GMTrade order accounts are forwarded without checking who owns them |
| <b>Severity</b> | High                                                                |
| <b>Status</b>   | Fixed                                                               |
| <b>Location</b> | gmtrade_adapter.rs                                                  |

### Description

Likelihood: Depends on GMTrade-side checks. If GMTrade trusts the caller for any of the writable accounts, this is directly exploitable. Impact: High. A keeper running a permissionless close could redirect the payout of the user's order to an attacker-controlled account.

GMTrade's `create_order_v2` takes a 25-account list with 11 writable slots. Among them are the escrows that hold collateral during the order's lifetime: `initial_collateral_token_escrow` at index 11, `final_output_token_escrow` at index 12, plus the long and short token escrows. The receiver Token program account at index 15 is the user's source ATA.



The Flipper adapter validates only the signer at index 0. Every other slot is taken from **remaining\_accounts** and forwarded directly into the CPI. The adapter comment argues this is safe because "GMTrade records the payout receiver inside the order at creation time, not in remaining\_accounts at fill time". That is technically right, but it is also the attack: the order records the receiver from **remaining\_accounts[12]** at creation. The keeper later honours whatever was written there.

A helper called **validate\_accounts\_from\_mapping** exists in the same file and would re-derive the market token against the on-chain **DexMarketMapping**. It is never called from **open\_position** or **close\_position**.

Permissionless paths like **execute\_stop\_out**, **execute\_trigger\_order**, and the limit-order close branch let any cranker build the full account list. If the cranker swaps in their own escrow at index 12, the order is created with that escrow as the payout target. When GMTrade's keeper later fills the order, the proceeds go where the cranker pointed them, not back to the user's vault.

This finding is marked as suspect because full exploitability depends on whether GMTrade itself re-derives the escrow PDAs server-side. If it does, the substitution fails. If it accepts what the caller passes, the substitution succeeds. The Flipper-side gap is unambiguous either way.

## Locations

- **programs/flipper-core/src/adapters/gmtrade\_adapter.rs**: L36 (writable index list)
- **programs/flipper-core/src/adapters/gmtrade\_adapter.rs**: L107 to 167 (open, only index 0 pinned)
- **programs/flipper-core/src/adapters/gmtrade\_adapter.rs**: L169 to 217 (close, same)
- **programs/flipper-core/src/adapters/gmtrade\_adapter.rs**: L39 to 54 (helper that is defined but never called)
- **tools/idl-cache/extracted/gmtrade\_create\_order.json** (account list)

## Suggested Fix

Inside **'open\_position'** and **'close\_position'**, re-derive every Flipper-controllable account against its canonical seeds and require equality:

- Index 11, 12, 13, 14: re-derive against GMTrade's escrow seeds. The exact seeds must come from gmsol-store v2; pin them from the IDL.
- Index 15: re-derive the user's vault USDC ATA from **'get\_associated\_token\_address(vault, mint)'** and require it.
- Index 3: re-derive against **'DexMarketMapping.market'**. The existing **'validate\_accounts\_from\_mapping'** helper does most of this already, just wire it into the open and close paths.

Mirror the same set of checks across both paths so a fix to one cannot

drift  
from the other.

## Proof of Concept

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_only_the_signer_slot_is_pinned() {
        // The adapter pins index 0 with 'validate_owner_pda' and nothing
        // else.
        // The writable list says every one of these is writable in the
        // CPI.
        let writable: Vec<usize> = CREATE_ORDER_WRITABLE.iter().copied().
            collect();
        assert_eq!(writable, vec![3, 4, 5, 6, 11, 12, 13, 14, 15, 21,
            22]);
        assert!(writable.contains(&12), "final_output_token_escrow at
            12");
        assert!(writable.contains(&11), "initial_collateral_token_escrow
            at 11");
        assert!(writable.contains(&15), "initial_collateral_token_source
            at 15");
    }

    #[test]
    fn test_close_signature_inherits_the_same_gap_as_open() {
        // A fix must add new account-derivation logic in the body of
        both
        // open_position and close_position. Pin both signatures so they
        // are caught together.
        let _open_fp: fn(
            u64, u8,
            crate::state::order_intent::Side,
            crate::state::order_intent::MarketType,
            u16, u64,
            &[AccountInfo],
            &[&[u8]],
            &[&[u8]],
        ) -> Result<CpiOpenResult> = open_position;

        let _close_fp: fn(
            u64,
            crate::state::order_intent::Side,
            crate::state::order_intent::MarketType,
            u16, u64,
            &[AccountInfo],
            &[&[u8]],
        ) -> Result<CpiCloseResult> = close_position;
    }
}
```

```

        &[&[u8]],
    ) -> Result<CpiCloseResult> = close_position;
}
}

```

## H-06: Liquidation may lock a user's positions on the DEX forever

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <b>H-06</b>     | Liquidation may lock a user's positions on the DEX forever |
| <b>Severity</b> | High                                                       |
| <b>Status</b>   | Fixed                                                      |
| <b>Location</b> | liquidate_position.rs: L170-176, L15-60                    |

### Description

Likelihood: Medium. Impact: High.

Flipper has three different ways to wind down a position, and they behave very differently. **execute\_stop\_out** actually closes the position on the DEX and then records it as closed (safe). **handle\_liquidation** only records a position as liquidated after it has checked that the DEX has already closed it (safe). **liquidate\_position** records the whole position as liquidated based only on the oracle price, without touching the DEX at all and without checking whether the DEX has closed anything. This is the broken one.

Once **liquidate\_position** stamps the position as Liquidated, every instruction that could close it refuses to act, because they all only work on positions that are still open. The user's position is still live on the DEX, but Flipper now treats it as finished. Since only Flipper's vault can sign the close on the DEX (the user cannot do it themselves), the funds are stuck. On mainnet there is no recovery tool, so they are stuck for good.

The instruction's account list contains only the oracle price feed. There is no DEX program and no DEX position account, so it physically cannot close anything on the DEX; it only changes Flipper's own records. It assumes that "the oracle crossed our liquidation price" means "the DEX has already liquidated the position", which is not true in general. Flipper and the DEX use different price thresholds and different oracles, so Flipper's liquidation fires first, while the DEX position is still open. The trigger gains nothing directly, so this is griefing or operational damage (likelihood Medium), but the user's loss is real and permanent.

## Suggested Fix

liquidate\_position must not stamp a position as liquidated (which blocks all closing) without making sure the DEX side is actually settled. In order of preference:

1. Remove liquidate\_position entirely. It is redundant: execute\_stop\_out already closes on the DEX and charges the penalty, and handle\_liquidation already does the DEX-verified bookkeeping. The only thing this instruction adds is an unsafe shortcut.
2. Make it require proof the DEX is already closed, the way handle\_liquidation does: take each leg's DEX account, check it is empty, and only then mark it liquidated.

## Proof of Concept

```
// liquidate_position.rs::tests
// Returns true if a close path would ACCEPT this status (allow action).
// Copied verbatim from each instruction's require!(matches!(...)) gate.
fn accepted_by_compound_close(s: PositionStatus) -> bool {
    matches!(s, PositionStatus::Open | PositionStatus::PartiallyOpen
              | PositionStatus::PartiallyFilled)
}
fn accepted_by_close_with_collateral(s: PositionStatus) -> bool {
    matches!(s, PositionStatus::Open | PositionStatus::PartiallyOpen
              | PositionStatus::PartiallyFilled)
}
fn accepted_by_trigger_order(s: PositionStatus) -> bool {
    matches!(s, PositionStatus::Open | PositionStatus::PartiallyOpen
              | PositionStatus::PartiallyFilled)
}
fn accepted_by_stop_out(s: PositionStatus) -> bool {
    matches!(s, PositionStatus::Open | PositionStatus::PartiallyOpen
              | PositionStatus::PartiallyFilled)
}
fn accepted_by_emergency_close(s: PositionStatus) -> bool {
    matches!(s, PositionStatus::Open | PositionStatus::PartiallyOpen
              | PositionStatus::PartiallyFilled | PositionStatus::
                PendingFill
              | PositionStatus::PendingClose)
}

#[test]
fn test_liquidate_orphans_dex_legs_no_close_path_repro() {
```

```

    // After liquidate_position runs, this is the terminal aggregate
    status.
    let status_after_liquidate = PositionStatus::Liquidated;

    // Every close path rejects Liquidated. The position's live
    // DEX legs (never closed by liquidate_position) can no longer be
    acted upon.
    assert(!(accepted_by_compound_close(status_after_liquidate)));
    assert(!(accepted_by_close_with_collateral(status_after_liquidate)));
    assert(!(accepted_by_trigger_order(status_after_liquidate)));
    assert(!(accepted_by_stop_out(status_after_liquidate)));
    assert(!(accepted_by_emergency_close(status_after_liquidate)));

    // handle_liquidation rejects any sub already in Liquidated, which is
    // exactly what liquidate_position sets every sub to.
    let sub_status_after_liquidate = SubPositionStatus::Liquidated;
    assert!(sub_status_after_liquidate == SubPositionStatus::Liquidated);

    // Conclusion: zero close paths remain.
}

```

H-07: A leftover failed leg lets a user drain their vault while a position is still open

|                 |                                                                                    |
|-----------------|------------------------------------------------------------------------------------|
| <b>H-07</b>     | A leftover failed leg lets a user drain their vault while a position is still open |
| <b>Severity</b> | High                                                                               |
| <b>Status</b>   | Fixed                                                                              |
| <b>Location</b> | handle_liquidation.rs: L44-51, L81-102                                             |

### Description

Likelihood: Medium (needs a specific but realistic setup, a position that ended up closed while leaving a leg that failed to fill). Impact: High.

Every user has a counter for how many positions they currently have open. The withdraw instruction relies on it: if the count is zero, it assumes the user has nothing at risk and lets them withdraw everything without checking margin.

**handle\_liquidation** records a position as liquidated after the DEX has already closed it, and reduces the counter when a position becomes fully wound down. The problem is that it can be tricked into reducing the same user's counter a second time, using a leg that was left behind in a "failed to fill" state. It checks the individual leg it is processing (rejecting only legs that are already

liquidated or already closed) but never checks the overall state of the position, and it does not reject a "failed to fill" leg. So a position that is already fully closed can be entered again through its leftover failed leg. Flipping that leg to liquidated makes the "all legs closed or liquidated" check pass, and the counter is reduced again for a position that was already counted once. Push the counter below the real number of open positions, ideally to zero, and the withdraw safety check switches off. The user then withdraws everything while a real position is still open on the DEX, and the protocol absorbs the loss.

### Suggested Fix

```
// Add a check at the start of the instruction so a position that is
  already
// finished cannot be entered again:
require!(
  matches!(
    position.status,
    PositionStatus::Open
    | PositionStatus::PartiallyOpen
    | PositionStatus::PartiallyFilled
    | PositionStatus::PendingFill
    | PositionStatus::PendingClose
  ),
  FlipperError::PositionNotActive
);
// This stops the second entry, because the position is already closed by
  then.
// As a second layer, also refuse to liquidate a "failed to fill" leg.
```

### Proof of Concept

```
// handle_liquidation.rs::tests
fn make_sub(dex: DexType, status: SubPositionStatus) -> SubPosition {
  SubPosition {
    dex, dex_position_account: Pubkey::default(), allocation_percent:
    50,
    size_usd: 25_000_000_000, status, closed_size_usd: 25_000_000_000
  },
}

#[test]
fn test_double_decrement_via_fillfailed_repro() {
  // Position A already resolved to Closed by timeout_async_fill:
  //   - Drift leg closed normally and ends Closed
  //   - Jupiter async leg times out and ends FillFailed
  // The counter was already decremented once at that point.
  // True open-position count should be 1 (position B still open).
  let mut subs = [make_sub(DexType::Drift, SubPositionStatus::Closed);
MAX_SUB_POSITIONS];
  subs[1] = make_sub(DexType::Jupiter, SubPositionStatus::FillFailed);
```

```

let sub_positions_len = 2usize;

// Sub-status guard at handle_liquidation.rs:44-51
let jupiter = &subs[1];
let passes_sub_guard = jupiter.status != SubPositionStatus::
Liquidated
    && jupiter.status != SubPositionStatus::Closed;
assert!(passes_sub_guard); // FillFailed passes (only Liquidated/
Closed excluded)

// No aggregate position.status gate exists in the handler.
subs[1].status = SubPositionStatus::Liquidated; // flip FillFailed to
Liquidated

let all_liquidated = subs[..sub_positions_len]
    .iter().all(|sp| sp.status == SubPositionStatus::Liquidated);
let all_closed_or_liquidated = subs[..sub_positions_len].iter().all(|
sp| {
    matches!(sp.status, SubPositionStatus::Closed | SubPositionStatus
::Liquidated)
});
assert!(!all_liquidated); // Drift is Closed, not Liquidated
assert!(all_closed_or_liquidated); // fires again on an already-
Closed position

// Second decrement on an already-Closed position
let mut active_positions_count: u8 = 1; // only position B is open
if all_liquidated || all_closed_or_liquidated {
    active_positions_count = active_positions_count.saturating_sub(1)
;
}
// counter driven to 0 while position B is still open;
// withdraw then skips all margin enforcement.
assert_eq!(active_positions_count, 0);
}

```

## H-08: Collateral over-withdrawal through handle\_withdraw

|                 |                                                    |
|-----------------|----------------------------------------------------|
| <b>H-08</b>     | Collateral over-withdrawal through handle_withdraw |
| <b>Severity</b> | High                                               |
| <b>Status</b>   | Fixed                                              |
| <b>Location</b> | withdraw.rs: L145                                  |

## Description

Likelihood: High (any user who has previously opened and closed at least one position can do it).  
Impact: High (the user empties the protocol-owned USDC vault while real, open positions remain on upstream DEXes, creating bad debt).

`handle_withdraw` increments the counter of processed positions even when a position is inactive (Closed, Liquidated, Settled). So a user with a single active position and any previously closed position account can supply only the closed account in `remaining_accounts`. The counter matches, and the required-margin total stays at zero. The entire vault can then be drained while the user's external DEX exposure is preserved.

The hole is the status filter sitting between the per-position checks and the margin accumulator. A Closed, Liquidated or Settled position bumps `checked_count` and continues, so it is counted as "processed" but never contributes to the required margin. After the loop, `checked_count >= active_count` holds ( $1 \geq 1$ ), the required margin is 0, and the transfer drains the vault. The active position is still live on the DEX. If it moves against the user, the protocol takes the loss as bad debt. If it moves in the user's favour, the user has effectively obtained free leverage while risking nothing in the (already emptied) vault.

## Suggested Fix

```
// Do not count inactive positions toward checked_count.
if !matches!(*status,
    PositionStatus::Open
    | PositionStatus::PartiallyFilled
    | PositionStatus::PartiallyOpen
    | PositionStatus::PendingFill
    | PositionStatus::PendingClose
) {
    continue; // just continue, do not bump checked_count
}
```

## Proof of Concept

```
// withdraw.rs::tests
// Replays the loop body + post-loop guards of handle_withdraw as they exist
// on this branch (the buggy behaviour where non-active statuses bump
// checked_count via continue).
fn simulate_buggy_withdraw(
    pairs: &[(PositionStatus, u128)], // (status, total_size_usd_6dec)
    active_count: usize,
    post_withdrawal_balance: u128,
    required_bps: u128,
) -> std::result::Result<(), &'static str> {
    let mut checked_count = 0usize;
    let mut total_required_margin: u128 = 0;
    for (status, size) in pairs {
        // assume the supplied PDAs are authentic and unique
```



```

        if !matches!(*status,
            PositionStatus::Open | PositionStatus::PartiallyFilled
            | PositionStatus::PartiallyOpen | PositionStatus::PendingFill
            | PositionStatus::PendingClose
        ) {
            checked_count += 1;
            continue; // counts inactive positions as processed (the bug)
        }
        let required_for_pos = size.checked_mul(required_bps).unwrap() /
10_000;
        total_required_margin = total_required_margin
            .checked_add(required_for_pos).unwrap();
        checked_count += 1;
    }
    if active_count > 0 {
        if checked_count < active_count { return Err("
InvalidRemainingAccounts"); }
        if post_withdrawal_balance < total_required_margin {
            return Err("WithdrawalExceedsMargin");
        }
    }
    Ok(())
}

#[test]
fn test_status_filter_bypass_repro() {
    // Attacker: one historic Closed PDA, active_positions_count = 1
    // (a real open position #1 with $50K notional, 5% maintenance is $2
    .5K).
    // Attacker supplies only the closed pair; drains the entire vault.
    let pairs = [(PositionStatus::Closed, 0u128)];
    let active_count = 1;
    let post_withdrawal_balance = 0u128; // drain everything
    let required_bps = 500u128;          // 5% maintenance margin

    let result = simulate_buggy_withdraw(
        &pairs, active_count, post_withdrawal_balance, required_bps);

    // withdraw succeeds even though position #1 was never validated.
    assert!(result.is_ok());
}

```

## H-09: Collateral over-withdrawal through `handle_withdraw_collateral`

|                 |                                                                            |
|-----------------|----------------------------------------------------------------------------|
| <b>H-09</b>     | Collateral over-withdrawal through <code>handle_withdraw_collateral</code> |
| <b>Severity</b> | High                                                                       |
| <b>Status</b>   | Fixed                                                                      |
| <b>Location</b> | <code>withdraw_collateral.rs</code> : L99-103, L131, L151-159              |

### Description

Likelihood: Medium (any user can produce the precondition, the exploit is a single transaction). Impact: High (collateral over-withdrawal leaving live DEX exposure undercollateralized).

`handle_withdraw_collateral` is the non-USDC sibling of `handle_withdraw`. It inherits the same flawed completeness check and adds a second, independent arithmetic error. Two distinct bugs:

Bug A: the same status-filter bypass as H-08, except this function additionally lacks the `position_id` dedup that `withdraw.rs` has, so the attack works even with one historic Closed position account replayed N times in `remaining_accounts`. There is also no aggregate margin assertion after the loop at all (unlike `withdraw.rs`, which at least checks the post-balance against the total required margin).

Bug B: the per-position margin gate `meets_initial_margin(post_withdrawal_balance, pos.total_size_usd, ...)` is called inside the loop with the same `post_withdrawal_balance` for every position. The collateral vault holds a single shared balance, but the check treats each position as if it had exclusive access to the full balance. So a vault balance that covers each position individually, but not their sum, passes anyway, and the user withdraws past the aggregate margin requirement.

### Suggested Fix

```
// withdraw_collateral.rs, a minimal patch covers both bugs:
let mut checked_count: usize = 0;
let mut total_required_margin: u128 = 0;
let mut seen_position_ids: [u64; 128] = [u64::MAX; 128];
let mut seen_count: usize = 0;
let mut i = 0;
while i + 1 < remaining.len() {
    // ... (PDA + user checks unchanged) ...

    // [1] dedup by position_id (copy from withdraw.rs)
    let mut is_dup = false;
    for s in 0..seen_count {
        if seen_position_ids[s] == pos.position_id { is_dup = true; break; }
    }
    require(!is_dup, FlipperError::InvalidRemainingAccounts);
    if seen_count < 128 { seen_position_ids[seen_count] = pos.position_id
```

```

; seen_count += 1; }

// [2] skip non-active WITHOUT bumping checked_count (fix A)
if !matches!(pos.status, PositionStatus::Open | PositionStatus::
PartiallyFilled
    | PositionStatus::PartiallyOpen | PositionStatus::PendingFill |
PositionStatus::PendingClose)
{ continue; }

// [3] accumulate aggregate margin instead of per-position check (fix
B)
let required_bps = (registry.maintenance_margin_bps as u64)
    + (config.initial_margin_buffer_bps as u64);
let required_for_pos = (pos.total_size_usd as u128)
    .checked_mul(required_bps as u128).ok_or(FlipperError::
MathOverflow)? / 10_000;
total_required_margin = total_required_margin
    .checked_add(required_for_pos).ok_or(FlipperError::MathOverflow)
?;
checked_count += 1;
}

```

## Proof of Concept

```

// withdraw_collateral.rs::tests
#[derive(Clone, Copy)]
struct PosRec { position_id: u64, status: PositionStatus, total_size_usd:
u64, mm_bps: u16 }

// Replays the current buggy loop body + post-loop guard.
fn simulate_buggy_withdraw_collateral(
    pairs: &[PosRec], active_count: usize,
    post_withdrawal_balance: u64, buffer_bps: u16,
) -> std::result::Result<(), &'static str> {
    let mut checked_count: usize = 0;
    for pos in pairs {
        // no dedup in current code; repeated position_id passes through.
        if !matches!(pos.status,
            PositionStatus::Open | PositionStatus::PartiallyFilled
            | PositionStatus::PartiallyOpen | PositionStatus::PendingFill
            | PositionStatus::PendingClose) {
            checked_count += 1;
            continue;
        }
        // Per-position check reuses the full post_withdrawal_balance
        each time.
        if !margin::meets_initial_margin(
            post_withdrawal_balance, pos.total_size_usd, pos.mm_bps,
            buffer_bps) {
            return Err("WithdrawalExceedsMargin");
        }
    }
}

```

```

        }
        checked_count += 1;
    }
    if active_count > 0 && checked_count < active_count {
        return Err("InvalidRemainingAccounts");
    }
    Ok(()) // no aggregate margin guard in current code
}

#[test]
fn test_solveny_bug_a_status_filter_bypass_repro() {
    // active_positions_count = 1 (real position open, not supplied).
    let pairs = [PosRec { position_id: 0, status: PositionStatus::Closed,
                          total_size_usd: 0, mm_bps: 500 }];
    assert!(simulate_buggy_withdraw_collateral(&pairs, 1, 0, 100).is_ok())
};

#[test]
fn test_solveny_bug_a_no_dedup_repeat_pda_repro() {
    // active_positions_count = 3; same Closed PDA supplied 3 times.
    let one = PosRec { position_id: 0, status: PositionStatus::Closed,
                      total_size_usd: 0, mm_bps: 500 };
    let pairs = [one, one, one];
    assert!(simulate_buggy_withdraw_collateral(&pairs, 3, 0, 100).is_ok())
};

#[test]
fn test_solveny_bug_b_shared_balance_repro() {
    // Two active positions, $50K each (5% mm + 1% buffer is $3K each,
    // $6K total).
    // Post-balance $3K covers each individually but not the sum.
    let pos = PosRec { position_id: 0, status: PositionStatus::Open,
                      total_size_usd: 50_000_000_000, mm_bps: 500 };
    let pos2 = PosRec { position_id: 1, ..pos };
    let pairs = [pos, pos2];
    assert!(simulate_buggy_withdraw_collateral(&pairs, 2, 3_000_000_000,
    100).is_ok());
}

```

## H-10: Unbound collateral Pyth feed lets a user underpay protocol fees

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <b>H-10</b>     | Unbound collateral Pyth feed lets a user underpay protocol fees |
| <b>Severity</b> | High                                                            |
| <b>Status</b>   | Fixed                                                           |
| <b>Location</b> | execute_trade_with_collateral.rs: L307, L310-313                |

### Description

Likelihood: High (a single pubkey swap in the client transaction, no privileged conditions, every non-stablecoin trade). Impact: Medium (indirect protocol-revenue loss, underpayment factor up to about 500x when a BTC-priced feed is substituted for SOL collateral).

The market Pyth feed at `remaining_accounts[0]` is validated against `MarketRegistry.pyth_feed_id`. The collateral Pyth feed at `remaining_accounts[1]` is consumed without any binding to `params.collateral_type`. A trader can set `collateral_type = Sol` but supply a BTC/USD feed. `scale_fee_to_collateral` then divides the USD fee by the inflated BTC price, so the SPL transfer moves a tiny fraction of the intended native amount, while treasury accounting still records the full USD amount. In effect the books say "\$25 collected" while the vault receives roughly \$0.05.

The root cause is an asymmetry between two adjacent reads: the market read is bound by a key check against the registry feed id, but the collateral read has no equivalent check. There is no `CollateralRegistry` on this branch, so there is no mint-to-feed mapping to bind against, and the handler also skips the confidence check for the collateral feed.

### Suggested Fix

```
// state/collateral.rs - hardcode the CollateralType to pyth_feed_id
mapping
impl CollateralType {
  pub fn pyth_feed_id(&self) -> [u8; 32] {
    match self {
      Self::Usdc | Self::Usdt => [0u8; 32], // unused: stablecoin
      short-circuit
      Self::Sol => SOL_PYTH_FEED_ID,
      Self::Wbtc => BTC_PYTH_FEED_ID,
      Self::Weth => ETH_PYTH_FEED_ID,
    }
  }
}

// execute_trade_with_collateral.rs:307 - add the binding + confidence
check
require!(
  col_pyth_account.key().to_bytes() == params.collateral_type.
  pyth_feed_id(),
  FlipperError::InvalidOraclePrice
```

```
);
crate::helpers::validate_pyth_confidence(
    col_price.price, col_price.conf, config.max_oracle_conf_bps)?;
```

## Proof of Concept

```
// execute_trade_with_collateral.rs::tests
#[test]
fn test_unbound_collateral_oracle_substituted_btc_for_sol() {
    // Collateral is SOL, but the trader passes BTC/USD as
    remaining_accounts[1].
    // scale_fee_to_collateral receives the BTC price for a SOL-decimals
    trade.
    // Buggy native fee = fee_usd6 * 10^9 / 100_000_000_000
    //                      = 2_500_000 * 1e9 / 1e11 = 25_000 lamports (~$0
    .005).
    // Truthful would be 12_500_000 lamports ($2.50). Underpayment factor
    : 500x.
    let native_attack = scale_fee_to_collateral(
        FEE_USD_6, &CollateralType::Sol, Some(BTC_USD_SUBSTITUTED)).
    unwrap();
    assert_eq!(native_attack, 25_000);

    let truthful = 12_500_000u64;
    let underpayment_factor = truthful / native_attack;
    assert!(underpayment_factor >= 500);
}
```

H-11: A partial close on Adrena is recorded as partial but the DEX has already flattened the whole position

|                 |                                                                                                       |
|-----------------|-------------------------------------------------------------------------------------------------------|
| <b>H-11</b>     | A partial close on Adrena is recorded as partial but the DEX has already flattened the whole position |
| <b>Severity</b> | High                                                                                                  |
| <b>Status</b>   | Fixed                                                                                                 |
| <b>Location</b> | adrena_adapter.rs, execute_close_with_collateral.rs                                                   |

## Description

Likelihood: High at the audited snapshot. Triggered by an ordinary user action. Impact: High. The position ends up in a state the protocol cannot clean up, with phantom margin requirements blocking

withdraws.

The Adrena close adapter accepts a `close_size_usd` argument with an underscore prefix, meaning the value is ignored. The instruction that goes out to Adrena always carries `percentage = ADRENA_FULL_CLOSE_BPS`, which is 100 percent. The adapter then returns `fully_closed: false`.

The handler in `execute_close_with_collateral` reads `fully_closed: false` to mean "the DEX took only what we asked for, this was a partial close" and writes `sp.closed_size_usd += close_size`. The `close_size` here is the partial amount the user requested. On the DEX side, however, Adrena has closed the entire position and sent every dollar of margin plus PnL back to the per-mint vault.

So the user requests 50 percent of a \$50k Adrena position. Adrena closes the full \$50k and returns \$50k of value. Flipper writes \$25k against `closed_size_usd`, flips the leg to `PartialClosed`, and the aggregate position becomes `PartiallyOpen`. The phantom residual \$25k now has no real DEX position behind it.

Any subsequent attempt to close the residual through Adrena reverts because the position account is gone. `close_aggregated_position` refuses because the position is not terminal. `withdraw_collateral` computes maintenance margin against the phantom residual and locks the portion of vault the user "should" be backing. The only way out is admin migration.

The handoff documentation claimed this was fixed. On the audited snapshot it was not fixed. A later commit on the same branch adds a real `close_percent` argument and computes `percentage = adrena_close_percentage(close_percent)`. That fix is the right shape.

## Locations

- `programs/flipper-core/src/adapters/adrena_adapter.rs`: L168 to 205 (close path on the audited snapshot, hardcodes 100 percent)
- `programs/flipper-core/src/adapters/cpi_interfaces/adrena.rs`: L100 (the `ADRENA_FULL_CLOSE_BPS` constant)
- `programs/flipper-core/src/instructions/execute_close_with_collateral.rs`: L355 to 369 (handler that records the requested partial size)

## Suggested Fix

```
Already applied on the branch in commit 22
f2af5e6b9ca5fcef9e62380309d5b467d86d92
("fix(dex): reconcile partial close accounting", 2026-06-09). The fix
threads
the user's 'close_percent' through the dispatcher into
'adrena_adapter::close_position' and uses it to build the close params:
```

```
percentage: adrena_close_percentage(close_percent)?
```

```
The audited snapshot 1b409f2 predates this commit and is still vulnerable
.
Make sure 22f2af5 is included in whichever revision goes to mainnet.
```

## Proof of Concept

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_adrena_close_params_hardcode_full_size_at_the_snapshot() {
        // The constant the adapter passes regardless of what the user
        // asked for.
        assert_eq!(ADRENA_FULL_CLOSE_BPS, 1_000_000);

        let params = ClosePositionParams {
            price: Some(100_000_000),
            oracle_prices: None,
            multi_oracle_prices: None,
            percentage: ADRENA_FULL_CLOSE_BPS,
        };
        assert_eq!(params.percentage, ADRENA_FULL_CLOSE_BPS);
    }

    #[test]
    fn test_handler_records_partial_while_adrena_closes_full() {
        let sp_size_usd: u64 = 50_000_000_000; // $50k position
        let mut sp_closed_size_usd: u64 = 0;
        let requested_close: u64 = 25_000_000_000; // user asks 50%

        // What the adapter actually returns.
        let cpi_result = CpiCloseResult { accounts_consumed: 0,
            fully_closed: false };

        // Replay handler branch.
        let actual_closed = if cpi_result.fully_closed {
            sp_size_usd.saturating_sub(sp_closed_size_usd)
        } else {
            requested_close
        };
        sp_closed_size_usd += actual_closed;

        // Books say half closed.
        assert_eq!(sp_closed_size_usd, 25_000_000_000);
        assert!(sp_closed_size_usd < sp_size_usd);

        // Reality on the DEX: 100% closed.
        let dex_actual_closed = sp_size_usd;
        let phantom_residual = dex_actual_closed - actual_closed;
        assert_eq!(phantom_residual, 25_000_000_000);
    }
}
```



## H-12: Phoenix close trusts the requested size instead of reading what actually filled

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| <b>H-12</b>     | Phoenix close trusts the requested size instead of reading what actually filled |
| <b>Severity</b> | High                                                                            |
| <b>Status</b>   | Fixed                                                                           |
| <b>Location</b> | phoenix_adapter.rs, execute_close_with_collateral.rs                            |

### Description

Likelihood: High at the snapshot. Any thin order book or out-of-range slippage bound produces a partial or zero fill. Impact: High. The protocol records the position as closed while the user's exposure stays live on the DEX.

The Phoenix close adapter returns `CpiCloseResult { fully_closed: false, accounts_consumed: remaining_accounts.len() }` unconditionally. It never reads the Phoenix subaccount after the CPI to find out what actually filled. The order packet sets `min_base_lots_to_fill = 0` and `min_quote_lots_to_fill = 0`, so Phoenix happily returns success on a zero-lot fill.

The handler then takes the "partial close" branch and writes `closed_size_usd += close_size` (the requested amount). The leg is marked Closed if the cumulative reaches the position size, and the aggregate transitions accordingly. In the stop-out path it is worse: `sp.closed_size_usd = sp.size_usd` is set unconditionally, and `active_positions_count` is decremented.

If Phoenix filled nothing, Flipper still thinks the user's position is closed. The trader's exposure stays live on the DEX. Combined with H-01 (no withdraw path), any margin that should have come back to the vault is permanently locked on the Phoenix side.

The handoff doc claims this was fixed in section 3.7 with post-CPI reconciliation. That fix exists on a later commit, where the close handler now calls `adapters::realized_sync_close_size` to compute `actual_closed` from the pre and post subaccount diff. At the audited snapshot, the reconciliation is not there.

### Locations

- `programs/flipper-core/src/adapters/phoenix_adapter.rs`: L177 to 232 (close returns `fully_closed: false` unconditionally)
- `programs/flipper-core/src/adapters/cpi_interfaces/phoenix.rs`: L171 to 172 (zero-fill is allowed by the packet)
- `programs/flipper-core/src/instructions/execute_close_with_collateral.rs`: L355 to 371 (handler records requested size)

## Suggested Fix

Already applied on the branch in commit 22

f2af5e6b9ca5fcef9e62380309d5b467d86d92

("fix(dex): reconcile partial close accounting", 2026-06-09). The close handler now calls 'adapters::realized\_sync\_close\_size' to compute the actual closed size from the pre and post Phoenix subaccount diff.

As a belt-and-braces guard alongside that fix, set 'min\_base\_lots\_to\_fill', equal to the requested base lots so a zero-fill bubbles up as a CPI revert.

The audited snapshot 1b409f2 predates this commit and is still vulnerable.

## Proof of Concept

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_phoenix_close_signature_does_not_take_the_expected_position()
    {
        // No room to thread the subaccount key needed for the pre and
        post diff.
        let _close_fp: fn(
            u64,
            crate::state::order_intent::Side,
            crate::state::order_intent::MarketType,
            u16, u64,
            &[AccountInfo],
            &[&[u8]],
        ) -> Result<CpiCloseResult> = close_position;
    }

    #[test]
    fn test_zero_fill_is_recorded_as_full_close() {
        let sp_size_usd: u64 = 10_000_000_000; // $10k position
        let mut sp_closed_size_usd: u64 = 0;
        let requested_close: u64 = 10_000_000_000; // user asks 100%

        // The adapter's actual return value at the audited snapshot.
        let cpi_result = CpiCloseResult { accounts_consumed: 0,
            fully_closed: false };

        // Handler branch.
        let actual_closed = if cpi_result.fully_closed {
            sp_size_usd.saturating_sub(sp_closed_size_usd)
        } else {
```

```

        requested_close
    };
    sp_closed_size_usd += actual_closed;

    // Books say fully closed.
    assert!(sp_closed_size_usd >= sp_size_usd);

    // Reality: Phoenix filled zero lots. All $10k of exposure is
    still live.
    let dex_actual_closed: u64 = 0;
    let residual = sp_size_usd - dex_actual_closed;
    assert_eq!(residual, 10_000_000_000);
}
}

```

## 2.3 Medium

M-01: Sour close lets the user redirect the proceeds away from the protocol vault

|                 |                                                                             |
|-----------------|-----------------------------------------------------------------------------|
| <b>M-01</b>     | Sour close lets the user redirect the proceeds away from the protocol vault |
| <b>Severity</b> | Medium                                                                      |
| <b>Status</b>   | Fixed                                                                       |
| <b>Location</b> | sour_adapter.rs, execute_close_with_collateral.rs                           |

### Description

Likelihood: Medium. The closer is the user themselves, so this is self-exploitation, not theft of someone else's funds. Impact: Low to Medium. Direct effect is fee evasion. Secondary effect is that vault accounting no longer reflects the user's true balance and breaks future margin checks.

The Sour close path runs three CPIs in sequence. The last one is a sweep that moves the closed-out USDC from the per-position conduit token account into the user's per-mint vault USDC ATA. The destination of that sweep is whatever account is at index 1 in **remaining\_accounts**. The adapter's account-pinning helper explicitly skips index 1 with the comment "we do not decode the SPL token account here" and a claim that swapping it would amount to "losing to themselves".

That claim holds on the open path, where index 1 is both source and destination of net-zero flows. It does not hold on the close path, where index 1 is only the destination of a one-way transfer. SPL Token checks the source's authority, not the destination. So the user can supply any USDC token account they own at index 1, and the close proceeds land there instead of in the canonical per-mint

vault.

The user is the one signing the close, so they are moving their own money. But this gives them two practical effects.

First, fee evasion. The close-side fee block checks `scaled_total > col_vault.amount` and silently skips fee collection when the vault does not have enough to cover the fee. By draining the close proceeds elsewhere, the user makes the vault balance zero just before the fee waterfall runs, and the guard kicks in. Fees are skipped on every close they route this way.

Second, the protocol's view of the user's effective collateral on later withdrawals becomes wrong. The PnL is not in the per-mint vault, so the vault residual the margin check sees is smaller than the user's actual liquidity. This is currently grief against the user themselves, but it sets up future composite vulnerabilities if any state-replay path is added.

### Locations

- `programs/flipper-core/src/adapters/sour_adapter.rs`: L202 to 212 (the deliberate skip of index 1)
- `programs/flipper-core/src/adapters/sour_adapter.rs`: L583 to 607 (the sweep that uses index 1 as destination)
- `programs/flipper-core/src/instructions/execute_close_with_collateral.rs`: L498 to 518 (the fee-skip guard)

### Suggested Fix

Pin index 1 to the canonical per-mint vault USDC ATA.

The cheapest option is to accept it as a typed Anchor account on the instruction context and constrain its address via the associated-token-program helper, against the vault PDA and the collateral mint:

```
seeds = associated_token::get_associated_token_address(vault, mint)
```

Alternatively, decode the SPL token account at index 1 inside the adapter and require its mint and owner. Mirror the same check on the open path for defence in depth.

### Proof of Concept

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_close_path_does_not_validate_the_sweep_destination() {
        assert_eq!(IDX_VAULT_USDC_ATA, 1);
    }
}
```

```

        assert_eq!(IDX_POSITION_VAULT, 2);
        assert_eq!(IDX_POSITION_VAULT_USDC, 3);
    }

    #[test]
    fn test_substituted_destination_is_a_different_token_account() {
        // SPL Transfer does not check destination authority, only the
        source.
        // The user can put their own ATA at index 1, the sweep will land
        there.
        let canonical = Pubkey::new_unique();
        let attacker_ata = Pubkey::new_unique();
        assert_ne!(canonical, attacker_ata);
    }

    #[test]
    fn test_fee_block_skips_collection_when_vault_balance_is_zero() {
        // Replay the guard from execute_close_with_collateral. If the
        proceeds
        // went elsewhere, the vault is empty and the fee is silently
        skipped.
        let scaled_total: u64 = 1_250_000;           // $1.25 fee
        let vault_after_close: u64 = 0;
        let fee_collected = if scaled_total > vault_after_close { 0 }
    else { scaled_total };
        assert_eq!(fee_collected, 0);
    }
}

```

M-02: Emergency close marks a position as closed while an async leg is still pending

|                 |                                                                                |
|-----------------|--------------------------------------------------------------------------------|
| <b>M-02</b>     | Emergency close marks a position as closed while an async leg is still pending |
| <b>Severity</b> | <b>Medium</b>                                                                  |
| <b>Status</b>   | <b>Fixed</b>                                                                   |
| <b>Location</b> | emergency_close_sub_position.rs, aggregated_position.rs                        |

### Description

Likelihood: Medium. Requires a tripped circuit breaker or a protocol halt, plus a multi-leg position with one sync leg and one async leg. Impact: Medium. The async leg becomes permanently stuck, the parent position cannot be cleaned up, and the user's active-position count drifts.

**emergency\_close\_sub\_position** is a permissionless instruction that lets anyone close a single leg of a position when the protocol is halted or the relevant DEX circuit breaker is active. After it marks the leg closed, it transitions the aggregate position based on the count of legs that are still "open".

The count uses **open\_sub\_position\_count**, which is filtered by **SubPosition::is\_open()**. That predicate matches only **Open** and **PartialClosed**. It does not match **PendingFill** or **PendingClose**. So a position that has one normal Adrena leg and one async GMTrade leg that is still awaiting confirmation gets counted as having one open leg, not two.

When the cranker closes the Adrena leg, the count drops to zero (the GMTrade leg is in **PendingFill** and not counted). The aggregate flips to **Closed**, and the active-position counter is decremented.

The GMTrade leg is still in **PendingFill** on Flipper's books. **confirm\_async\_fill** and **timeout\_async\_fill** both require the aggregate to be in {**PendingFill**, **PendingClose**}, so they reject the now-Closed aggregate. **close\_aggregated\_position** refuses because **has\_inflight\_sub\_position** (which does include **PendingFill**) still returns true. The only way out is the admin migration tool. The user's active-position count is also one lower than the reality, so they can open more positions while still holding the phantom-pending leg.

Scenario:

1. The user opens a position split as Adrena \$25k and GMTrade \$25k. The GMTrade leg goes **PendingFill**. The aggregate is **PendingFill**.
2. The admin trips the Adrena circuit breaker.
3. A cranker calls **emergency\_close\_sub\_position** on the Adrena leg.
4. The Adrena leg is marked **Closed**. The open count is now zero because **PendingFill** is not counted. The aggregate is set to **Closed**. The active-position count goes from 1 to 0.
5. The GMTrade leg is still **PendingFill**. No instruction will accept the position. The rent on the parent PDA cannot be reclaimed.

## Locations

- `programs/flipper-core/src/instructions/emergency_close_sub_position.rs`: L172 to 194 (transition that does not check inflight legs)
- `programs/flipper-core/src/state/aggregated_position.rs`: L114 to 119 (**is\_open** definition)
- `programs/flipper-core/src/state/aggregated_position.rs`: L261 to 272 (**has\_inflight\_sub\_position**)

## Suggested Fix

Before flipping the aggregate to 'Closed', also require there is no inflight async leg. The 'has\_inflight\_sub\_position' predicate already exists and is

used by 'close\_aggregated\_position' for the same reason.

```
let open_count = position.open_sub_position_count();
let inflight = position.has_inflight_sub_position();
if open_count == 0 && !inflight {
    position.status = PositionStatus::Closed;
    user_account.active_positions_count =
        user_account.active_positions_count.saturating_sub(1);
} else if open_count < position.sub_positions_len {
    position.status = PositionStatus::PartiallyOpen;
}
```

Make sure 'confirm\_async\_fill' and 'timeout\_async\_fill' accept a 'PartiallyOpen' aggregate as well so the pending leg can be unwound through the normal path afterwards.

## Proof of Concept

```
#[cfg(test)]
mod audit_tests {
    use crate::constants::MAX_SUB_POSITIONS;
    use crate::state::aggregated_position::*;
    use crate::state::order_intent::DexType;
    use anchor_lang::prelude::Pubkey;

    fn open_sub(dex: DexType, size: u64) -> SubPosition {
        SubPosition {
            dex, dex_position_account: Pubkey::default(),
            allocation_percent: 50, size_usd: size,
            status: SubPositionStatus::Open,
            closed_size_usd: 0, fill_request_ts: 0,
        }
    }

    fn pending_fill_sub(dex: DexType, size: u64) -> SubPosition {
        let mut sp = open_sub(dex, size);
        sp.status = SubPositionStatus::PendingFill;
        sp.fill_request_ts = 1;
        sp
    }

    #[test]
    fn test_emergency_close_orphans_a_pending_fill_leg() {
        let mut subs: [SubPosition; MAX_SUB_POSITIONS] =
            std::array::from_fn(|_| open_sub(DexType::Adrena, 0));
        subs[0] = open_sub(DexType::Adrena, 25_000_000_000);
        subs[1] = pending_fill_sub(DexType::GMTTrade, 25_000_000_000);

        let mut pos = AggregatedPosition {
```

```

        // ... boilerplate fields ...
        sub_positions: subs,
        sub_positions_len: 2,
        status: PositionStatus::PendingFill,
        // ...
    };

    // Replay the emergency-close transition on sub index 0 (Adrena).
    {
        let sp = &mut pos.sub_positions[0];
        sp.closed_size_usd = sp.size_usd;
        sp.status = SubPositionStatus::Closed;
    }
    pos.total_size_usd = pos.total_remaining_size();
    let open_count = pos.open_sub_position_count();
    if open_count == 0 {
        pos.status = PositionStatus::Closed;
    } else if open_count < pos.sub_positions_len {
        pos.status = PositionStatus::PartiallyOpen;
    }

    // PendingFill is not counted by is_open, so the count is zero
and
    // the aggregate flips to Closed even though one leg is still
pending.
    assert_eq!(open_count, 0);
    assert_eq!(pos.status, PositionStatus::Closed);
    assert!(matches!(pos.sub_positions[1].status, SubPositionStatus::
PendingFill));

    assert!(pos.has_inflight_sub_position());
    assert!(
        !matches!(
            pos.status,
            PositionStatus::PendingFill | PositionStatus::PendingClose
        )
    );
}
}

```



M-03: Phoenix legs report they consumed the entire account tail, breaking any compound trade where Phoenix is not last

|                 |                                                                                                                  |
|-----------------|------------------------------------------------------------------------------------------------------------------|
| <b>M-03</b>     | Phoenix legs report they consumed the entire account tail, breaking any compound trade where Phoenix is not last |
| <b>Severity</b> | Medium                                                                                                           |
| <b>Status</b>   | Fixed                                                                                                            |
| <b>Location</b> | phoenix_adapter.rs, execute_trade_with_collateral.rs                                                             |

### Description

Likelihood: Medium. Any router that lists Phoenix anywhere except last triggers it. Impact: Medium. The transaction reverts after the Phoenix CPI has already executed, wasting compute and emitting partial events.

Phoenix's `place_market_order` takes a variable number of accounts because the trailing arenas are book-specific. The Flipper adapter declares it consumed `remaining_accounts.len()` of the tail it was handed, not its own footprint. The caller increments `remaining_idx` by that amount, lands past the end of the buffer, and any later leg receives an empty slice and reverts on the length check.

Worse, the Phoenix CPI itself forwards that full tail to `invoke_signed`, so the next DEX's accounts ride along into Phoenix as more orderbook entries. Phoenix's own validation rejects them, but the resulting failure looks like a Phoenix bug to anyone debugging.

This is contained as long as the off-chain router always lists Phoenix as the last leg of a compound trade. There is no on-chain guard. A future change to the router, a permissionless limit-fill cranker, or a hand-built multi-leg intent can violate the ordering, and the program will not catch it before the Phoenix CPI runs.

Scenario:

1. The router builds a compound trade with allocations Phoenix 50 percent and Adrena 50 percent. The tail has 200 accounts: about 30 for Phoenix, the rest for Adrena.
2. The dispatcher hands the full 200-account tail to the Phoenix adapter.
3. Phoenix CPI runs (or fails on its own, separately). The adapter returns `accounts_consumed: 200`.
4. The dispatcher advances `remaining_idx` by 200. The Adrena leg receives an empty slice and reverts. The whole transaction reverts atomically.

### Locations

- `programs/flipper-core/src/adapters/phoenix_adapter.rs`: L171 (open returns full tail length)
- `programs/flipper-core/src/adapters/phoenix_adapter.rs`: L230 (close returns full tail length)

- `programs/flipper-core/src/instructions/execute_trade_with_collateral.rs`: L563 (caller advances `remaining_idx`)

## Suggested Fix

Parameterise the Phoenix adapter with the exact account count the caller intends to use. Forward only that prefix to `'invoke_signed'`, return that count as `'accounts_consumed'`, and reject when `'remaining_accounts.len()'` is less than the supplied count.

If pinning a fixed count per market is too restrictive, enforce on-chain that Phoenix is always the last leg by checking the allocation index at the dispatch site.

## Proof of Concept

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_phoenix_returns_the_entire_remaining_tail_length() {
        // Both open and close build their result with
        // 'accounts_consumed: remaining_accounts.len()'. Compare against
        // 'GMTRADE_CREATE_ORDER_ACCOUNTS = 25', which is the right shape
        :
        // a fixed integer the adapter knows about.
        let open_res = CpiOpenResult {
            position_account: anchor_lang::prelude::Pubkey::new_unique(),
            accounts_consumed: 200,
        };
        let close_res = CpiCloseResult { accounts_consumed: 200,
fully_closed: false };
        assert_eq!(open_res.accounts_consumed, 200);
        assert_eq!(close_res.accounts_consumed, 200);
    }

    #[test]
    fn test_following_leg_receives_an_empty_slice() {
        use super::super::cpi_interfaces::phoenix::
        PHOENIX_MARKET_ORDER_MIN_ACCOUNTS;

        let total = 200usize;
        let mut remaining_idx = 0usize;

        // Phoenix says it consumed the whole tail.
        remaining_idx += total;
        let next_slice_len = total.saturating_sub(remaining_idx);
        assert_eq!(next_slice_len, 0);
    }
}
```

```

        // Phoenix really only needs the minimum prefix, not 200 accounts
        .
        assert!(PHOENIX_MARKET_ORDER_MIN_ACCOUNTS < total);
    }
}

```

## M-04: Withdraw silently sees zero Sour collateral and grief-blocks the user

|                 |                                                                       |
|-----------------|-----------------------------------------------------------------------|
| <b>M-04</b>     | Withdraw silently sees zero Sour collateral and grief-blocks the user |
| <b>Severity</b> | Medium                                                                |
| <b>Status</b>   | Fixed                                                                 |
| <b>Location</b> | withdraw_collateral.rs, adapters/custody.rs                           |

### Description

Likelihood: Medium. Triggered by any user with a leveraged Sour leg where the DEX-side collateral matters for the margin check. Impact: Low to Medium. Pure grief, the user cannot withdraw amounts that should be available. No money is lost.

The withdraw instruction tries to use the DEX-side collateral to compute the user's true effective collateral. For Sour, the actual USDC margin sits in a separate **TraderAccount** account (discriminator byte 4), not in the Position account itself. The custody decoder for Sour needs both accounts in the per-leg slice.

The withdraw instruction parses **remaining\_accounts** as a strict sequence of [AggregatedPosition, MarketRegistry] pairs. The first non-position account in any pair slot fails the Anchor deserialize check and the whole call reverts. So there is no room to thread a **TraderAccount** through. The only free trailing slot is already used for the optional collateral price feed when collateral is not USDC.

When the Sour decoder cannot find a **TraderAccount** in the slice, it falls through to **.unwrap\_or(0)** and silently returns zero collateral for that leg. The withdraw effective-collateral total now misses every dollar the user has on Sour. The margin check rejects withdrawals that should pass.

This is grief, not theft, but it also breaks the stated SUM-2 contract for Sour. And on the stop-out and trigger paths the same silent-zero default cuts the other way: a leg looks healthier than it really is and can be stopped out at prices that should still be safe.

### Locations

- `programs/flipper-core/src/instructions/withdraw_collateral.rs`: L183 to 277 (the rigid pair-loop)

- `programs/flipper-core/src/adapters/custody.rs`: L192 to 203 (`decode_sour` returns `unwrap_or(0)`)

## Suggested Fix

Pick one of these, the first is cleanest.

1. Move the Sour `'usdc_collateral'` onto Flipper's own `'SubPosition'` struct, cached at open and refreshed on confirm-fill and partial-close. The withdraw path then reads it from the position itself with no extra account juggling.
2. Extend the pair-loop to accept an optional Sour-only third slot per leg, so the `'TraderAccount'` can travel alongside.

Separately, replace `'.unwrap_or(0)'` inside `'decode_sour'` with an explicit error. Today a missing `'TraderAccount'` and a real zero balance look identical to callers, which is the wrong default given that the collateral is the whole point of the decoder.

## Proof of Concept

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_pair_loop_does_not_have_room_for_a_trader_account() {
        // 'AggregatedPosition' uses Anchor's 8-byte discriminator. Sour'
        // 'TraderAccount' uses a single-byte discriminator (= 4). They
        // cannot
        // share a structural slot in the pair-loop, and there are no
        // extra
        // slots per leg.
        const TA_DISC: u8 = 4;
        assert!(TA_DISC != 0);
    }

    #[test]
    fn test_sour_collateral_collapses_to_zero_when_account_is_absent() {
        // 'decode_sour' uses '.unwrap_or(0)'. Missing 'TraderAccount' is
        // indistinguishable from a zero balance.
        let no_ta_found: u64 = 0;
        assert_eq!(no_ta_found, 0);
    }
}
```

```
}
```

## M-05: Adrena close path lets the closer pick the rebate receiver

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <b>M-05</b>     | Adrena close path lets the closer pick the rebate receiver |
| <b>Severity</b> | Medium                                                     |
| <b>Status</b>   | Fixed                                                      |
| <b>Location</b> | adrena_adapter.rs                                          |

### Description

Likelihood: Medium if Adrena does not enforce the binding. Low otherwise. Impact: Medium. The rebate that was meant for the registered builder ends up at the closer's chosen account.

The Adrena open path correctly passes the registered builder pubkey into Adrena's open params at line 101 of the adapter. The close path does the opposite: it accepts a **builder** argument with a leading underscore, meaning it is ignored. The doc comment says builder attribution on the close side is "account-driven, the caller supplies the builder's referrer\_profile".

That referrer\_profile sits at index 12 of the close account list, and the adapter does include index 12 in its writable set, but it never compares **remaining\_accounts[12].key()** to the builder pubkey from the registry. Whether the substitution actually pays off depends on whether Adrena itself binds the profile to the position's stored referrer. If it does, the close just reverts on a mismatch. If it accepts any valid **ReferrerProfile** PDA, the closer pockets the rebate.

This is marked as suspect because the answer lives in Adrena's program, not in Flipper. The Flipper-side gap is unambiguous either way, and the FLIP 1-1 comment chain in **adapters/mod.rs** specifically warns against the pattern of "trust the DEX to validate". This is the one place in the codebase that violates that.

### Locations

- `programs/flipper-core/src/adapters/adrena_adapter.rs`: L101 (open path, correct)
- `programs/flipper-core/src/adapters/adrena_adapter.rs`: L167 to 244 (close path, **\_builder** ignored)
- `programs/flipper-core/src/adapters/adrena_adapter.rs`: L213 (index 12 in the writable set)

## Suggested Fix

Inside the Adrena close adapter, when the builder argument is set, require the account at index 12 to match:

```
if let Some(b) = _builder {
    require!(
        remaining_accounts[12].key() == b.builder_pubkey,
        FlipperError::InvalidRemainingAccounts
    );
}
```

If Adrena wraps the builder pubkey in a 'ReferrerProfile' PDA, derive that PDA from 'b.builder\_pubkey' against the Adrena program and compare to the derived address. Drop the underscore on the parameter once a real check is in place so the intent is obvious to future readers.

## Proof of Concept

```
#[cfg(test)]
mod tests {
    use super::*;
    use crate::adapters::BuilderTarget;
    use crate::adapters::CpiCloseResult;
    use crate::state::order_intent::{MarketType, Side};

    #[test]
    fn
    test_adrena_close_signature_takes_a_builder_argument_but_ignores_it()
    {
        // The signature does accept Option<BuilderTarget>. A fix must
        read
        // it inside the body. Pin the signature so a future fix that
        adds
        // a runtime check is visible alongside.
        let _close_fp: fn(
            u64,
            Side,
            MarketType,
            u16,
            u64,
            &[AccountInfo],
            &[&[u8]],
            &[&[u8]],
            Option<BuilderTarget>,
        ) -> Result<CpiCloseResult> = close_position;
    }
}
```

```

#[test]
fn test_index_12_is_in_the_close_writable_set() {
    // Index 12 is 'referrer_profile'. It is writable, and forwarded
into
    // the CPI from 'remaining_accounts[12]', but there is no
equality
    // check against the registry-bound builder pubkey.
    const CLOSE_WRITABLE_AUDIT_PIN: &[usize] = &[1, 2, 4, 5, 6, 7, 8,
9, 10, 11, 12];
    assert!(CLOSE_WRITABLE_AUDIT_PIN.contains(&12));
}
}

```

## M-06: USD close fee transferred as native token units, causing overcharge

|                 |                                                                      |
|-----------------|----------------------------------------------------------------------|
| <b>M-06</b>     | USD close fee transferred as native token units, causing overcharge  |
| <b>Severity</b> | Medium                                                               |
| <b>Status</b>   | Fixed                                                                |
| <b>Location</b> | execute_close_with_collateral.rs: L307, L311, L339, L374, L401, L412 |

### Description

Likelihood: Medium (requires the admin to set `close_fee_dbps > 0`, which defaults to 0; once enabled, every non-stablecoin close triggers it). Impact: Medium (direct over-transfer of user collateral. With WBTC at \$100K and 8 decimals, a \$1.25 intended fee pulls about \$1,250 of real value, and a thin vault can be fully drained. Treasury bookkeeping silently diverges from real flows).

This is the mirror image of H-08 / H-09 on the close side. The handler computes the close fee in USD (6 decimals) via `calculate_fee`, then passes that number directly to `token::transfer` without going through `scale_fee_to_collateral`. The transfer's amount parameter is in native token units, so a USD-6 numerator becomes a native debit. The open path does the conversion correctly. The close path forgot the helper. Stablecoin collateral (6 decimals) works only by coincidence.

### Suggested Fix

```

// Same helper as H-10. Three changes in execute_close_with_collateral:
// 1. Add pyth_price_feed to the account context, bind it via
//   require!(pyth_price_feed.key().to_bytes() == params.collateral_type
//     .pyth_feed_id())
//   and run validate_pyth_confidence.

```

```
// 2. Read oracle_price from the feed (mirror
    execute_trade_with_collateral.rs:285-298).
// 3. Wrap every transfer-side fee in
//   scale_fee_to_collateral(amount_usd6, &collateral_type, Some(
//     oracle_price))?
//   at the referral, insurance and treasury transfers.
```

## Proof of Concept

```
// execute_close_with_collateral.rs::tests
fn scale_fee_to_collateral_sim(
    fee_usd_6dec: u64, collateral: &CollateralType, oracle: Option<u64>,
) -> u64 {
    if collateral.is_stablecoin() { return fee_usd_6dec; }
    let oracle = oracle.expect("non-stable needs oracle");
    let decimals = collateral.decimals() as u32;
    let numerator = (fee_usd_6dec as u128) * 10u128.pow(decimals);
    (numerator / oracle as u128) as u64
}

#[test]
fn test_close_fee_native_unit_mismatch_sol_undercharge() {
    // SOL: 9 decimals at $200. Intended fee $1.25 = 1_250_000 USD-6.
    // Buggy: 1_250_000 treated as lamports. Real USD = 1.25M * $200 / 1
    // e9 = $0.25.
    let fee_usd_6: u64 = 1_250_000;
    let usd_value_buggy_6 = (fee_usd_6 as u128) * 200_000_000 / 1
    _000_000_000;
    assert_eq!(usd_value_buggy_6, 250_000); // $0.25

    let amount_fixed = scale_fee_to_collateral_sim(
        fee_usd_6, &CollateralType::Sol, Some(200_000_000));
    assert_eq!(amount_fixed, 6_250_000); // 0.00625 SOL ~ $1.25
    let usd_value_fixed_6 = (amount_fixed as u128) * 200_000_000 / 1
    _000_000_000;
    assert_eq!(usd_value_fixed_6, 1_250_000); // $1.25 intent

    // Buggy path undercharges 5x here (and over-charges for high-value
    // WBTC).
    assert_eq!(usd_value_fixed_6 / usd_value_buggy_6, 5);
}
```



## M-07: Unit mismatch in margin check blocks legitimate withdrawals

|                 |                                                             |
|-----------------|-------------------------------------------------------------|
| <b>M-07</b>     | Unit mismatch in margin check blocks legitimate withdrawals |
| <b>Severity</b> | Medium                                                      |
| <b>Status</b>   | Fixed                                                       |
| <b>Location</b> | withdraw_collateral.rs: L82, L151-159                       |

### Description

Likelihood: High (any normal-sized non-USDC vault triggers it). Impact: Low (legitimate withdrawals are blocked even when the vault massively over-collateralises the position, no funds are lost).

`handle_withdraw_collateral` calls `meets_initial_margin(post_withdrawal_balance, pos.total_size_usd, ...)`. The first argument is the vault balance in native token units (9 decimals for wSOL, 8 for WBTC). The second is the position size in USD with 6 decimals. The function compares them as if they were in the same unit, with no oracle interaction and no decimals normalisation. Only 6-decimal stablecoins happen to line up. Every higher-decimal collateral (SOL, WBTC, WETH) is compared wrong, so honest withdrawals are rejected even when the vault is worth far more than the required margin. The correct helper exists elsewhere in the codebase but was not called here.

### Suggested Fix

```
// helpers/margin.rs, convert native to USD-6 before comparing
pub fn collateral_native_to_usd6(
    native_amount: u64, collateral: &CollateralType,
    oracle_price_usd6_per_unit: Option<u64>,
) -> Result<u64> {
    if collateral.is_stablecoin() { return Ok(native_amount); } // 6-dec
    == USD-6
    let price = oracle_price_usd6_per_unit.ok_or(FlipperError::InvalidOraclePrice)?;
    require!(price > 0, FlipperError::InvalidPrice);
    let decimals = collateral.decimals() as u32;
    let value_usd6 = (native_amount as u128)
        .checked_mul(price as u128).ok_or(FlipperError::MathOverflow)?
        / 10u128.pow(decimals);
    require!(value_usd6 <= u64::MAX as u128, FlipperError::MathOverflow);
    Ok(value_usd6 as u64)
}
// Then gate the withdraw on the USD-6 value, not the raw native balance.
```

## Proof of Concept

```
// withdraw_collateral.rs::tests
#[test]
fn test_wrong_units_dos_normal_wbtc_user() {
    // 1 WBTC vault = 10^8 native units (~$100_000 real value).
    // $50_000 position needs required margin $3_000 = 3_000_000_000 USD
    // -6.
    // Current code compares native (10^8) to USD-6 required (3e9)
    // directly.
    // 10^8 >= 3e9 is false, so the honest withdraw is blocked.
    let post_balance_native: u64 = 100_000_000;
    let position_size_usd6: u64 = 50_000_000_000;
    let passes_buggy = margin::meets_initial_margin(
        post_balance_native, position_size_usd6, 500, 100);
    assert!(!passes_buggy);
}
```

M-08: Cranker-supplied slippage overrides the protocol cap, enabling worst-fill stop-outs

|                 |                                                                                     |
|-----------------|-------------------------------------------------------------------------------------|
| <b>M-08</b>     | Cranker-supplied slippage overrides the protocol cap, enabling worst-fill stop-outs |
| <b>Severity</b> | Medium                                                                              |
| <b>Status</b>   | Fixed                                                                               |
| <b>Location</b> | execute_stop_out.rs: L173-177                                                       |

### Description

Likelihood: Medium. Impact: Medium.

`handle_execute_stop_out` accepts a caller-controlled `params.max_slippage_bps` and uses it whenever it is non-zero, without comparing it to `protocol_config.max_slippage_bps`. The value flows into `compute_acceptable_close_price`, which has no internal cap. A keeper acting as the cranker can widen the accepted-fill window to almost the full collateral and capture the spread between the fair price and the floor as MEV. The trigger-order path, by contrast, uses the configured cap directly with no caller override.

## Suggested Fix

```
// execute_stop_out.rs:173-177, cap to the configured maximum
let slippage = if params.max_slippage_bps > 0 {
    core::cmp::min(params.max_slippage_bps, config.max_slippage_bps)
} else {
    config.max_slippage_bps
};
// Optionally reject params.max_slippage_bps > config.max_slippage_bps
outright.
```

## Proof of Concept

```
// execute_stop_out.rs::tests
#[test]
fn test_slippage_param_unbounded_bug_repro() {
    use crate::adapters::compute_acceptable_close_price;
    use crate::state::order_intent::Side;

    let config_slippage: u16 = 500;          // 5% protocol-wide cap
    let param_slippage: u16 = u16::MAX;     // 65_535 bps (~655%) from the
    cranker

    // Current selector (execute_stop_out.rs:173-177): no cap against
    config.
    let effective = if param_slippage > 0 { param_slippage } else {
    config_slippage };
    assert_eq!(effective, u16::MAX);

    // saturating_sub(10_000, 65_535) = 0, limit price becomes 0, any
    fill accepted.
    let oracle: u64 = 100_000_000;
    let limit = compute_acceptable_close_price(oracle, effective, Side::
    Long);
    assert_eq!(limit, 0);

    // With the fix (cap to config) the limit would be a sane ~$95.
    let fixed = core::cmp::min(param_slippage, config_slippage);
    assert_eq!(compute_acceptable_close_price(oracle, fixed, Side::Long),
    95_000_000);
}
```

## M-09: Partial fill finalized as fully closed in the async close branch

|                 |                                                                  |
|-----------------|------------------------------------------------------------------|
| <b>M-09</b>     | Partial fill finalized as fully closed in the async close branch |
| <b>Severity</b> | Medium                                                           |
| <b>Status</b>   | Fixed                                                            |
| <b>Location</b> | confirm_async_fill.rs: L144-153                                  |

### Description

Likelihood: Medium. Impact: Medium.

When a keeper confirms an async close, the handler trusts the reported `filled_size` for the closed amount but never checks whether it actually covers the remaining size. It sets `closed_size_usd = filled_size`, flips the status to Closed unconditionally, and the recompute loop then drops the sub-position from the position size because Closed is neither open nor pending. The remaining exposure on the DEX is left orphaned in Flipper's view. This is asymmetric with the open branch in the same function, which correctly treats a partial open as a reduced size that stays open. There is no lower-bound check on `filled_size`, and the `PartialClosed` status (which already exists and is handled correctly elsewhere) is never written here.

### Suggested Fix

```
// confirm_async_fill.rs:144-153, mirror the open-path semantics
let was_pending_close = sp.status == SubPositionStatus::PendingClose;
let sp_mut = &mut position.sub_positions[idx];
if was_pending_close {
    sp_mut.closed_size_usd = sp_mut.closed_size_usd
        .checked_add(params.filled_size).ok_or(FlipperError::MathOverflow)
        ?;
    sp_mut.status = if sp_mut.closed_size_usd >= sp_mut.size_usd {
        SubPositionStatus::Closed
    } else {
        SubPositionStatus::PartialClosed
    };
} else {
    sp_mut.status = SubPositionStatus::Open;
    sp_mut.size_usd = params.filled_size;
}
```

### Proof of Concept

```
// confirm_async_fill.rs::tests
#[test]
fn test_partial_close_finalized_as_full_repro() {
    // Drift sync leg already closed ($25K). Jupiter async leg
    PendingClose ($25K).
```

```

    let mut pos = make_position(vec![
        (DexType::Drift, 25_000_000_000, SubPositionStatus::Closed),
        (DexType::Jupiter, 25_000_000_000, SubPositionStatus::
PendingClose),
    ]);
    pos.sub_positions[0].closed_size_usd = pos.sub_positions[0].size_usd;
    pos.status = PositionStatus::PendingClose;
    pos.total_size_usd = 25_000_000_000;

    // Keeper reports a partial fill: only $5K of the $25K requested.
    let filled_size: u64 = 5_000_000_000;
    let idx = 1usize;
    assert!(filled_size <= pos.sub_positions[idx].size_usd); // upper-
bound passes

    let sp_mut = &mut pos.sub_positions[idx];
    sp_mut.closed_size_usd = filled_size;           // bug: not += and not
bounded below
    sp_mut.status = SubPositionStatus::Closed;      // bug: unconditional

    // $20K of exposure is still live on Jupiter, but the leg is terminal
Closed.
    let residual = pos.sub_positions[idx].size_usd - pos.sub_positions[
idx].closed_size_usd;
    assert_eq!(residual, 20_000_000_000);

    // The recompute loop then drops the residual from the aggregate size
.
    let mut recomputed: u64 = 0;
    for i in 0..pos.sub_positions_len as usize {
        let sp = &pos.sub_positions[i];
        if sp.is_open() || matches!(sp.status,
SubPositionStatus::PendingFill | SubPositionStatus::
PendingClose) {
            recomputed += sp.size_usd.saturating_sub(sp.closed_size_usd);
        }
    }
    pos.total_size_usd = recomputed;
    assert_eq!(pos.total_size_usd, 0); // believes there is no open
exposure
}

```

## M-10: Referral fee transfer breaks all non-USDC trades for referred users

|                 |                                                                     |
|-----------------|---------------------------------------------------------------------|
| <b>M-10</b>     | Referral fee transfer breaks all non-USDC trades for referred users |
| <b>Severity</b> | <b>Medium</b>                                                       |
| <b>Status</b>   | <b>Fixed</b>                                                        |
| <b>Location</b> | <code>execute_trade_with_collateral.rs</code> : L256-265, L339-351  |

### Description

Likelihood: High (fires on every non-USDC `execute_trade_with_collateral` for a user who has a referrer set). Impact: Low (denial of service, no funds lost).

The referral-fee mechanism was designed for the USDC-only flow and was not generalised when non-USDC collateral support was added. The non-USDC trade path validates the referrer vault against `[VAULT_SEED, referrer_pubkey]`, which is the referrer's USDC vault. It then tries to transfer the referral fee from the user's `collateral_vault` (holding SOL, WBTC, WETH or USDT) into that USDC vault. The SPL Token program rejects a transfer when the source and destination mints differ, so the whole transaction reverts. The validation and the transfer disagree about which vault should receive the fee, and there is no non-USDC referrer vault wired into the instruction. A referred user simply cannot open a non-USDC position until the referrer is cleared or they switch to USDC.

### Suggested Fix

```
// Option A: derive the referrer's per-mint collateral vault and use that
let (expected_vault, _) = Pubkey::find_program_address(
    &[COLLATERAL_VAULT_SEED, referrer_pubkey.as_ref(), vault_mint.as_ref()],
    ctx.program_id,
);
require!(referrer_vault.key() == expected_vault, FlipperError::InvalidReferrerAccount);

// Option B: keep payout in USDC, but skip referral on non-USDC for now
let has_referrer = user_account.referrer.is_some()
    && params.collateral_type.is_stablecoin();
```

### Proof of Concept

```
// execute_trade_with_collateral.rs::tests
#[test]
fn test_referral_vault_pda_derives_usdc_for_non_usdc_trade_repro() {
    use crate::constants::{COLLATERAL_VAULT_SEED, VAULT_SEED};
    let program_id = Pubkey::new_unique();
    let referrer_pubkey = Pubkey::new_unique();
    let sol_mint = Pubkey::new_unique();
```

```

// The handler validates against the USDC vault PDA regardless of the
trade mint.
let (expected_referrer_vault_usdc, _) = Pubkey::find_program_address(
    &[VAULT_SEED, referrer_pubkey.as_ref()], &program_id);

// The truthful per-mint referrer vault is a different address.
let (truthful_sol_vault, _) = Pubkey::find_program_address(
    &[COLLATERAL_VAULT_SEED, referrer_pubkey.as_ref(), sol_mint.
as_ref()],
    &program_id);

// They differ, so the transfer source (collateral_vault, non-USDC)
and the
// validated destination (USDC vault) have mismatched mints, so SPL
revert.
assert_ne!(expected_referrer_vault_usdc, truthful_sol_vault);
}

```

# CONCLUSION

The audit of the Flipper Perps Aggregator revealed **23 findings**: 1 Critical, 12 High and 10 Medium severity issues. **All 23 have been fixed** on the `fix/mainnet` branch and independently re-verified against the head commit listed in the Project Audit Log.

The Flipper Perps Aggregator is a genuinely interesting piece of engineering. Splitting a single user order across several perpetual DEXes atomically, keeping collateral in protocol-owned vaults, and reconciling per-DEX quirks under one aggregated position is a hard design problem, and the code-base solves it cleanly. The team engaged with every finding in detail and responded quickly.

The Critical treasury misconfiguration was closed by the per-mint withdraw refactor so accumulated protocol revenue is again reachable. The Phoenix cluster was closed by wiring the real deposit and withdraw margin lifecycle, pinning the trader subaccount to the leg's stored account before the reduce-only CPI, deriving the subaccount from a caller-supplied index so two positions can no longer share one, and routing the builder rebate through the Flight proxy so the configured builder actually receives it. GMTrade escrow slots and payout targets are now explicitly bound to their canonical ATAs, with unused slots pinned to the gmsol None-sentinel. Sour and Adrena rebate destinations are pinned to the registered profiles from the builder registry. The emergency close path no longer terminalises an inflight leg. And the earlier round of Highs and Mediums, from withdraw over-drain to the async-close reconciliation, remain closed and are verified to still be closed on the current tree.

In our view the protocol's security posture has meaningfully improved over the course of the engagement, and the aggregator is ready to move ahead with mainnet rollout on the conditions the team already documented (notably the Rise-side operational grant for Phoenix subaccounts, which is an off-chain coordination step rather than a code gap). Security is materially better than at the audited snapshot.



# ABOUT BUGBLOW

BugBlow is a security firm specializing in comprehensive audits and penetration testing for Web3 and traditional applications. Founded in 2024, BugBlow helps projects identify and eliminate vulnerabilities before they reach production, whether the code runs on-chain or off-chain.

Our team comprises security engineers and researchers with deep expertise in smart contract internals, backend architecture, cryptographic protocols, and application security. We combine manual review with automated tooling and AI-assisted analysis to deliver thorough, high-signal audit reports.

## Why BugBlow

- **Full-Stack Coverage:** We audit both on-chain smart contracts (Solidity, Solana, FunC) and off-chain application code (APIs, backends). Most vulnerabilities live at the boundary between the two, and we cover both sides.
- **Penetration Testing:** Beyond code review, we simulate real-world attack scenarios against live and staging environments to uncover issues that static analysis alone cannot find.
- **Adversarial Mindset:** Every audit includes Proof-of-Concept exploits for critical findings. We do not just flag theoretical risks, we demonstrate impact.
- **Developer-Friendly Process:** We stay in direct communication throughout the engagement, share preliminary findings early, and work with your team to verify fixes before the final report.

## Our Services

- **Smart Contract Audits:** Security assessment of on-chain code across EVM, TON, and Solana ecosystems. Includes manual review, automated analysis, and formal verification where applicable.
- **Off-Chain Audits:** Security review of server-side applications, APIs, microservices, and infrastructure code. Covers authentication, authorization, race conditions, input validation, data integrity, and business logic flaws.
- **Penetration Testing:** Black-box and gray-box testing of web applications, APIs, and infrastructure. Simulates external and internal threat actors to identify exploitable attack paths.
- **Security Consulting:** Architecture review, threat modeling, and security best practices guidance for teams building in Web3 and beyond.

## Contact Information

**Web**      <https://bugblow.com>

**GitHub**    <https://github.com/BugBlow/audits>

**X**          <https://x.com/BugBlow>